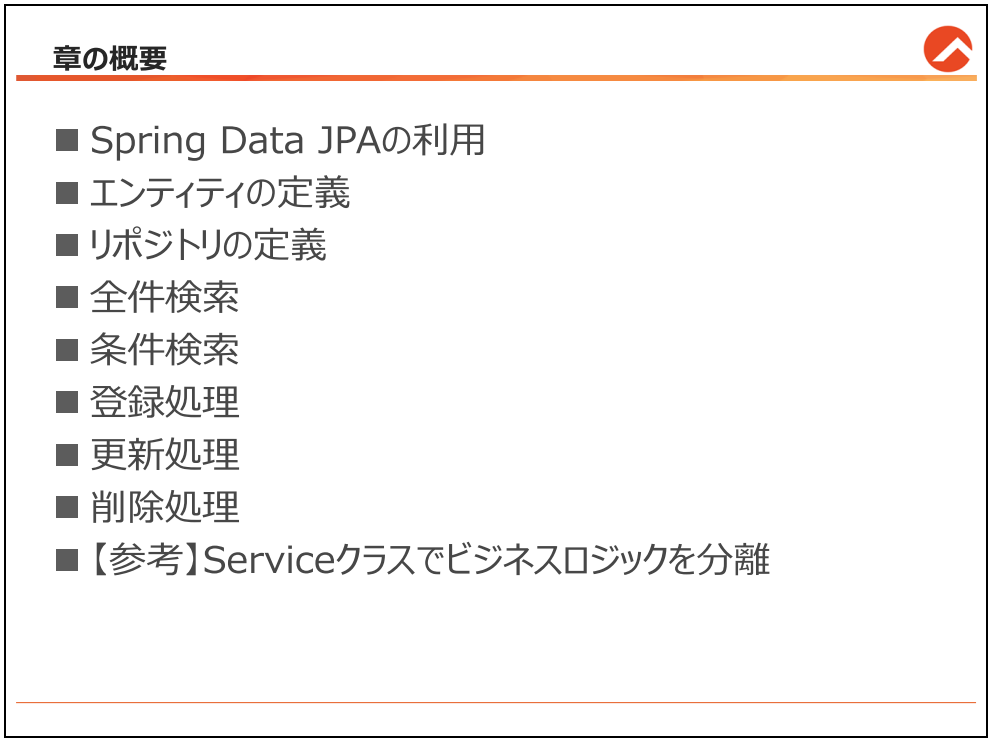
3

# 第3章 Spring Data JPA

## 章の目標

- Spring Data JPA の基本概念を理解できる
- エンティティとリポジトリを実装できる
- 全件検索・条件検索ができる
- 登録・更新・削除ができる

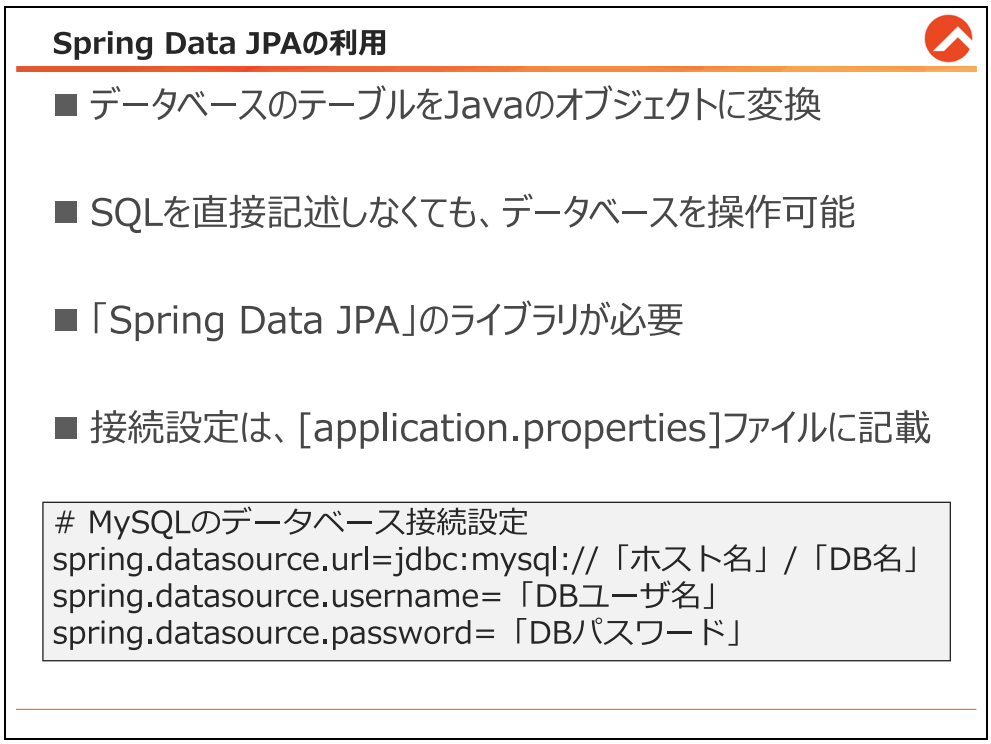
# 章の概要




第3 章では、Spring Data JPA の活用方法に焦点を当てて、Spring とデータベースを連携するWeb アプリケーションを構築する方法を学びます。

初めに、Spring Data JPA とは何か、そしてその利用のメリットについて理解を深めます。続いて、データを表現するためのエンティティの定義方法について説明し、データへのアクセスを抽象化するリポジトリの作成技術を学習します。さらに、データベースから情報を全件検索する方法、さまざまな条件を用いた検索方法についてもご説明します。その後、データの登録、更新、削除といったCRUD 操作の基本的なプロセスについて学ぶことで、Spring Data JPAを用いた効率的なデータ管理手法をマスターしていきましょう。

## 3.1. Spring Data JPA の利用




### Spring Data JPA とは

Web アプリケーションの開発では、データの保存や取得のためにデータベースとの連携が不可欠です。一般的に、データベースと連携するプログラミングは、以下のようにある程度パターン化されています。

**「データベースへの接続→データの検索→結果を取得→データベースからの切断」**

このようなパターンは一見シンプルに思えますが、Spring Framework などのツールを使わないと、膨大な量のコードを書く必要があります。

Spring Framework は、このような繰り返し行われる作業を効率化することに強みがあり、さまざまな支援ツールを提供しています。本書では、特にデータベース操作を簡単にできる「Spring Data JPA(Jakarta Persistence API)」に注目します。

通常、データベースに命令を出すためには、SQL という特別な言語を使う必要がありますが、**Spring Data JPA を使うと、予めデータベース情報をJava のオブジェクトに変換することで、テーブルの1 行のデータをJava の1 つのインスタンスとして扱うことができます。**

そのおかげで、**SQL 文を書かなくても、データベースの操作をすることが可能になります。**この節では、Spring Data JPA を使って、データを効果的に管理する方法について学びます。

## 【参考】サンプルデータベースの定義

本書では、以下のデータベースの情報を基にします。

| | |
|---|---|
| RDBMS 名 | MySQL8.0 |
| データベース名 | spring_sample |
| ユーザ名 | spring_sample_user |
| パスワード | trainocate |

3 章では、[Product]テーブルのみ使用します。

### <Product テーブル>

| No | 論理名称 | 物理名称 | データ型 | 桁数 | 制約 | 備考 |
|---|---|---|---|---|---|---|
| 1 | 商品番号 | id | INT | – | PRIMARY KEY AUTO_INCREMENT | 自動採番 |
| 2 | 商品名 | name | VARCHAR | 255 | – | – |
| 3 | 価格 | price | INT | – | – | – |

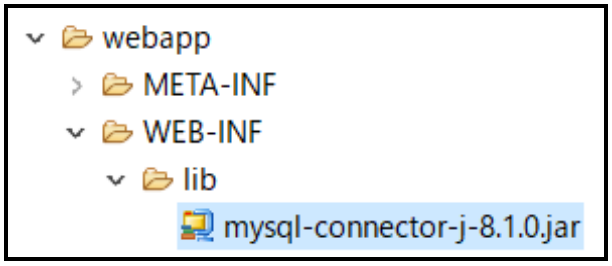
[product]テーブルには、9 行のデータが格納されています。

| 商品番号 | 商品名 | 価格 |
|---|---|---|
| 1 | スマートテレビ | 30000 |
| 2 | ゲーミングモニター | 20000 |
| 3 | ノートパソコン | 50000 |
| 4 | 2 ドア冷蔵庫 | 50000 |
| 5 | コンパクト冷蔵庫 | 15000 |
| 6 | ワインクーラー | 30000 |
| 7 | 全自動洗濯機 | 25000 |
| 8 | 乾燥機能付き洗濯機 | 40000 |
| 9 | 洗濯乾燥機 | 60000 |

## 【参考】既存のJDBC 接続(Java+MySQL)方法と問題

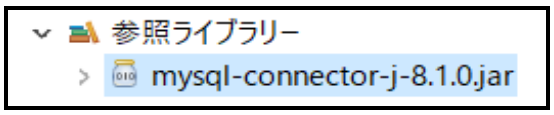
既存のJava データベースプログラミングにおいての、MySQL との接続方法を説明します。

1. 事前にmysql-connector-j.jar ファイルを入手し、プロジェクトのビルドパスに追加する










2. クラスに接続情報を記載する

```
String jdbcUrl =
      "jdbc:mysql://localhost/spring_sample?serverTimezone=Asia/
Tokyo";
String userId = "spring_sample_user";
String password = "trainocate";
```

3. DriverManager.getConnction(「DB URL」,「ユーザ名」,「パスワード」)メソッドを実行

4. SQL を実行するための処理を記載(例外処理必須)

```
try (Connection con =
            DriverManager.getConnection(jdbcUrl, userId, password))
{
            //データベース処理
      } catch (SQLException e) {
            e.printStackTrace();
      }
```

既存のJDBC 接続方法にはいくつかの欠点があります。そして、Spring Data JPA はその欠点を大きく改善しています。以下は、その比較表です。

| 要素 | JDBC | Spring Data JPA |
| --- | --- | --- |
| 外部ライブラリの依存 | JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要があ る。ダウンロードやビルドパスの設定など の追加作業が伴う。 | Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 |
| ハードコーディング | 接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。 | 接続情報は外部の設定ファイル (application.properties)に記載するため、 セキュリティが向上し、環境に応じた変更も 容易。 |
| 冗長なコード | 接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。 | JPA リポジトリインターフェースを使用するこ とで、冗長なコードを大幅に削減。よりシン プルで読みやすい。 |
| 例外処理の必要性 | DriverManager.getConnectionメソッドが 接続失敗時にSQLException をスローする ため、追加の例外処理コードが必要。 | RuntimeExceptionを使用するため、明示 的な例外処理が不要になる場合が多いの で、コードがシンプル。 |
| リソース管理 | 開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。 | リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 |

Spring Data JPA は、よりシンプルで安全、そして効率的なデータアクセスを提供することで、データベースプログラミングを容易にします。

## 【作業手順①】Spring Data JPA と、MySQL ドライバーの導入

第1 章でご説明した通り、Maven を利用すると簡単にライブラリを追加できます。

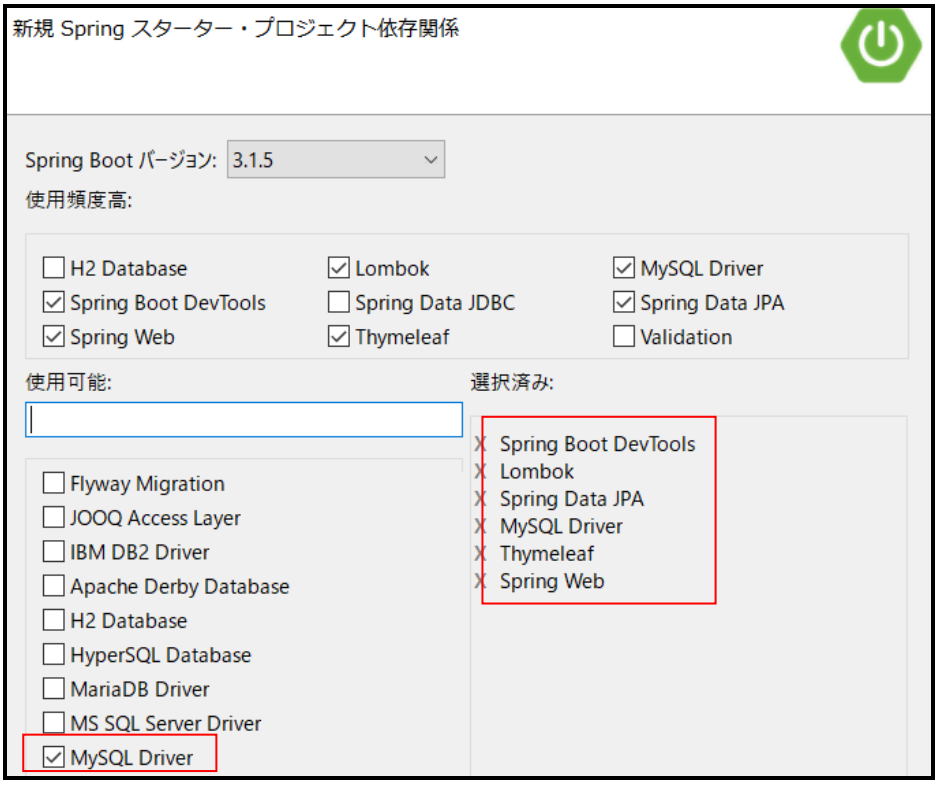
第3 章で必要なライブラリは、**「Spring Data JPA」と、「MySQL Driver」、「Lombok」**です。

または、Spring プロジェクトを作成する時、同時にインストールしたいライブラリを選択する方法もあります。

新規でプロジェクトを作成する際は、必要なライブラリを必ず導入してから進めてください。

本章の依存定義は次のとおりです。各バージョンはSpring Boot 4.0.2の親POMに任せます。MySQL 8以降を起動し、付録の手順で本文用の spring_sample を準備してから、配布設定を確認してください。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```


新規 Spring スターター・プロジェクト依存関係
Spring Boot バージョン: 3.1.5
使用頻度高:
H2 Database
Lombok
MySQL Driver
Spring Boot DevTools
Spring Data JDBC
Spring Data JPA
Spring Web
Thymeleaf
Validation
使用可能:
選択済み:
Flyway Migration
JOOQ Access Layer
IBM DB2 Driver
Apache Derby Database
H2 Database
HyperSQL Database
MariaDB Driver
MS SQL Server Driver
MySQL Driver
Spring Boot DevTools
Lombok
Spring Data JPA
MySQL Driver
Thymeleaf
Spring Web

## 【作業手順②】接続情報の設定

Spring Data JPA を使う場合、**データベースとの接続情報は、クラスに記載する必要はなく、「application.properties」ファイルに記載をします。**そうすると、全クラスがこの設定ファイルを参照することになりますので、管理がしやすくなります。

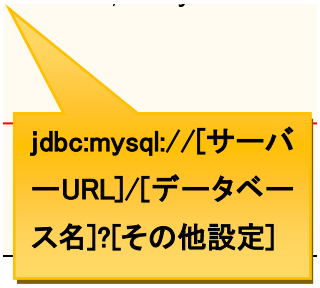



```
spring.thymeleaf.cache=false

# MySQL のデータベース接続設定
spring.datasource.url=jdbc:mysql://localhost/spring_sample?
serverTimezone=Asia/Tokyo
spring.datasource.username=spring_sample_user
spring.datasource.password=trainocate

# JPA/Hibernate の追加設定（開発環境ではお勧め）
spring.jpa.show-sql=true
```

spring.datasource.url の値は1行で記述します。紙面で折り返されていても、ファイル内では途中に改行を入れません。MySQLのポートを3306以外に変更した場合は、localhost:ポート番号 の形で実際の接続先に合わせます。




追記した各設定は以下の情報を表しています。

**spring.datasource.url→データベースの URL(サーバー名/データベース名が含まれます)**

**spring.datasource.username→データベースに接続するユーザ名**

**spring.datasource.password→データベースに接続するためのパスワード**

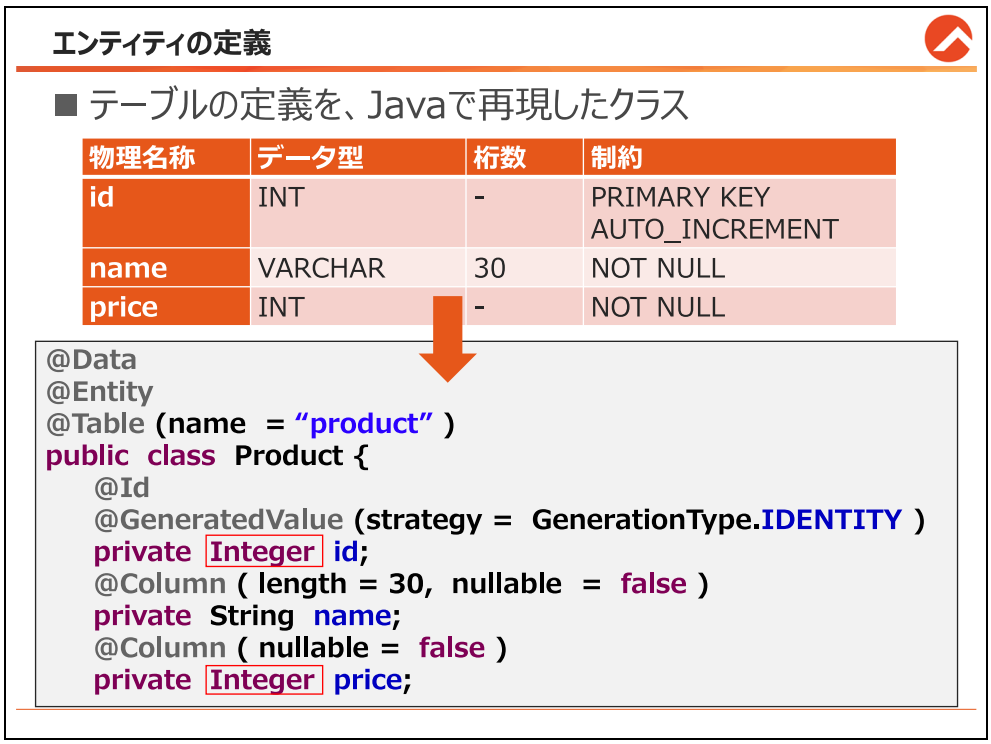
**spring.jpa.show-sql=true →生成したSQL 文をログで確認可能**

最後の1 行は、本番環境では使うべきではありませんが、開発時は役に立つ設定です。

この設定は、あくまでMySQL 専用の書き方であり、Oracle やPostgreSQL などのRDBMS と接続する場合は、別途調査が必要です。

application.properties を保存したら、アプリケーションを停止して再起動し、接続結果を確認します。依存定義も変更した場合は、[Maven]→[プロジェクトの更新]を実行してください。設定ファイルを変更するたびに install を実行する必要はありません。

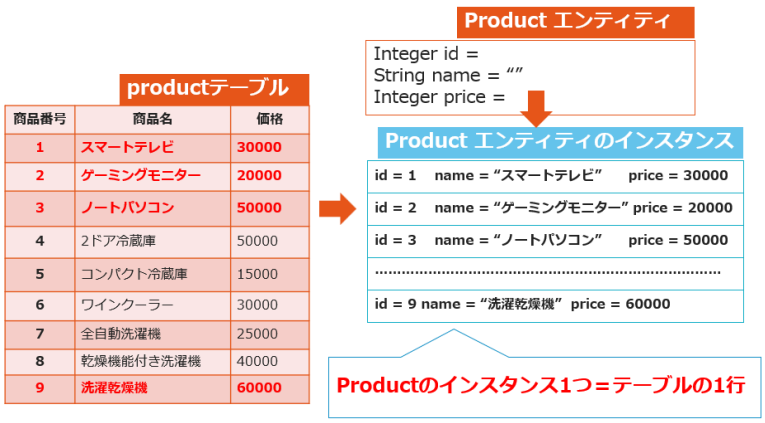
## 3.2. エンティティの定義



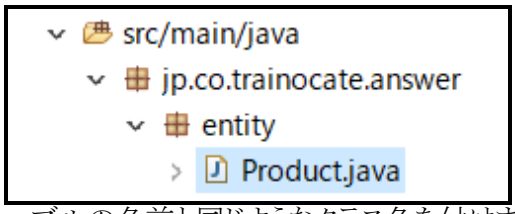

### エンティティ(Entity)とは？

Spring Data JPA を使用する際には、**データベース内の各テーブルをJava のクラスで表現**することが重要です。このクラスを「エンティティ」と呼びます。エンティティは実質的にデータベースのテーブルの各行をオブジェクトとして扱うためのものです。つまり、**テーブルの1 行がエンティティの1 インスタンス（オブジェクト）に対応します。**

エンティティの作成を学ぶことは、データベースをプログラムで扱うための最初のステップです。例として、もし「product」という名前のテーブルがある場合、それに対応するエンティティを「Product」というクラスとして作成します。このクラスには、テーブルの列に対応するフィールドが含まれ、データベースの行の情報を保持できるようになります。



### Entity クラスの作成




Entity クラスは、通常テーブルの名前と同じようなクラス名を付けます。

Controller クラスを、controller パッケージに作成したのと同じく、Entity クラスは、entity パッケージの中に作成します。

### 【参考】<Product テーブル>

| No | 論理名称 | 物理名称 | データ型 | 桁数 | 制約 | 備考 |
|---|---|---|---|---|---|---|
| 1 | 商品番号 | id | INT | - | PRIMARY KEY AUTO_INCREMENT | 自動採番 |
| 2 | 商品名 | name | VARCHAR | 30 | NOT NULL | - |
| 3 | 価格 | price | INT | - | NOT NULL | - |

それでは、Product テーブルを、Java のクラスで再現してみます。理解のため、全体のソースコードをお見せする前に、作業順でご説明します。

### 【作業手順①】テーブルの列名と型をJava のフィールドで表現する

基本的に、JavaBean クラスを作る感覚と似ていますので、product テーブルで使われるデータを、一つずつ定義していきましょう。

**…/entity/Product.java**

```
@Entity
@Table(name = "product")
public class Product {
    private Integer id;
    private String name;
    private Integer price;
}
```

このコードにおいて、@Entity アノテーションはこのクラスがエンティティクラスであることを示し、@Table アノテーションは実際のテーブル名を指定します。その後、テーブルの各列を型に合わせて定義します。この例では、id、name、price が該当します。

フィールドの型についてですが、int ではなく、Integer の型が推奨されます。その理由は、データベースの int 列には、null を格納できても、Java の int 型には null を入れることができないためです。

そして、JavaBean と同様にデータのやり取りを行うクラスなので、当然 Getter&Setter メソッドや、初期化のためのコンストラクタ、より完成度を上げるため toString メソッド、equals メソッド、hashcode メソッドを定義するのが理想です。

しかし、それを全て書くと、開発効率が落ちるため、lombokライブラリの@Dataアノテーションを付けると便利です。

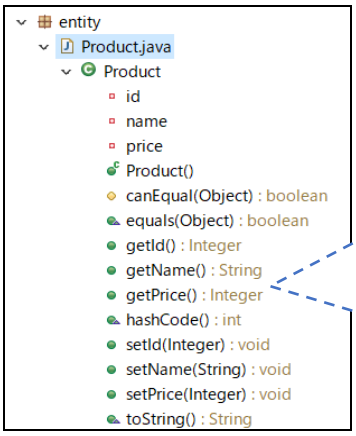
**【作業手順②】Lombok の@Data アノテーションを付与**

**…/entity/Product.java**

```
import lombok.Data;

@Entity
@Table(name = "product")
@Data
public class Product {
    private Integer id;
    private String name;
    private Integer price;
}
```

**@Data アノテーションを付与するだけで、Getter&Setter、コンストラクタ、toString、hashcode、equals メソッドが自動生成**されますので、非常に便利になります。




**@Data を付けた後、Eclipse 上で確認してみると、いつの間にかメソッドが生成されていることを確認できます。**

**【作業手順③】制約をアノテーションで表現**

次は、**制約についても、定義**する必要があります。

**…/entity/Product.java**

```
@Id //Primary Key 制約
@GeneratedValue(strategy = GenerationType.IDENTITY)//auto_increment
制約
private Integer id;

@Column(length = 30, nullable = false)    //桁数30, NOT NULL 制約
private String name;

@Column(nullable = false) //NOT NULL 制約
private Integer price;
```

**Primary Key 制約は、「@Id」アノテーションで表現します。普通の列については、「@Column」を付けます。**

新規登録をするたびに、自動的に番号が割り当てられる、自動採番の AUTO_INCREMENT制約は、「@GeneratedValue(strategy = GenerationType.IDENTITY)」で表現します。

入力値の長さや、NOT NULL 制約の有無なども、表現できます。

完成された Product クラスは、下記となります。

**…/entity/Product.java**

```
 1:package jp.co.trainocate.answer.entity;
 2:
 3:import jakarta.persistence.Column;
 4:import jakarta.persistence.Entity;
 5:import jakarta.persistence.GeneratedValue;
 6:import jakarta.persistence.GenerationType;
 7:import jakarta.persistence.Id;
 8:import jakarta.persistence.Table;
 9:import lombok.Data;
10:
11:@Data
12:@Entity
13:@Table(name = "Product")
```

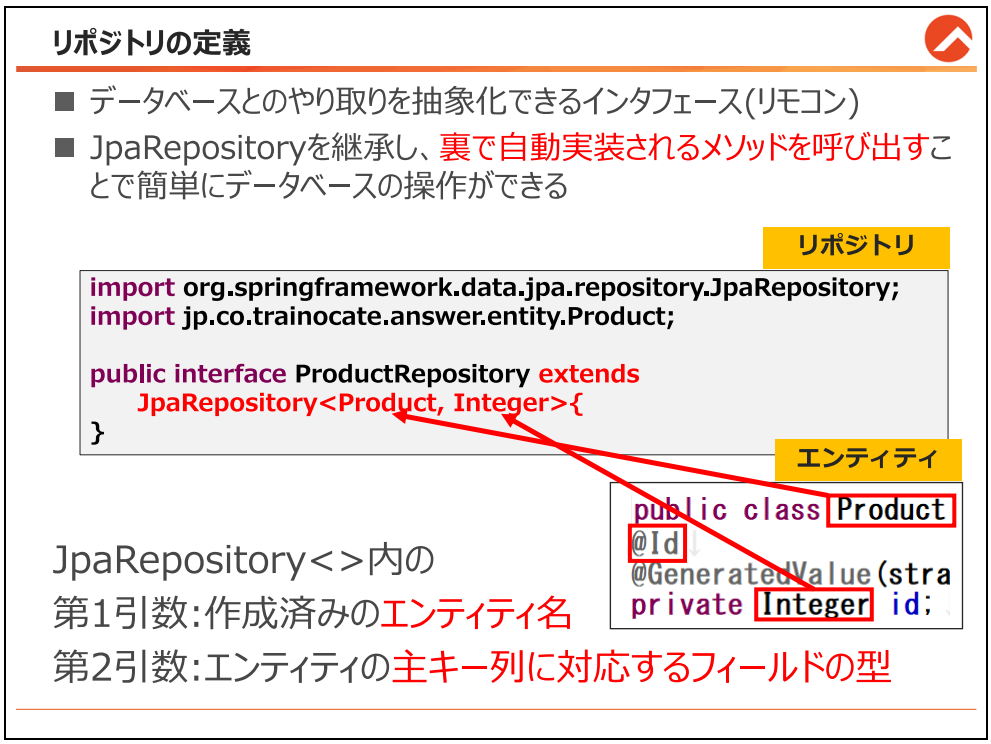
```
14:public class Product {
15:
16:    @Id
17:    @GeneratedValue(strategy = GenerationType.IDENTITY)
18:    private Integer id;
19:
20:    @Column(length = 30, nullable = false)
21:    private String name;
22:
23:    @Column(nullable = false)
24:    private Integer price;
25:
26:    // lombok がコンストラクタと以下のメソッドを自動生成
27:    //Getter()&Setter()
28:    //equals()
29:    //hashCode()
30:    //toString()
31:}
```

以上が、エンティティクラスの作成方法でした。エンティティは、全く言語が異なる、データベースを Java の世界に再現したものです。

データベースの各行はエンティティの Java オブジェクトとして扱われ、SQL を直接書くことなくデータベースとのやり取りが可能になります。

今回は一般的なエンティティクラスの書き方をご説明しましたが、Oracle などの異なる RDBMS の独自の機能を再現する際は、書き方が変わる場合がありますので、それぞれ調査が必要です。

## 3.3. リポジトリの定義



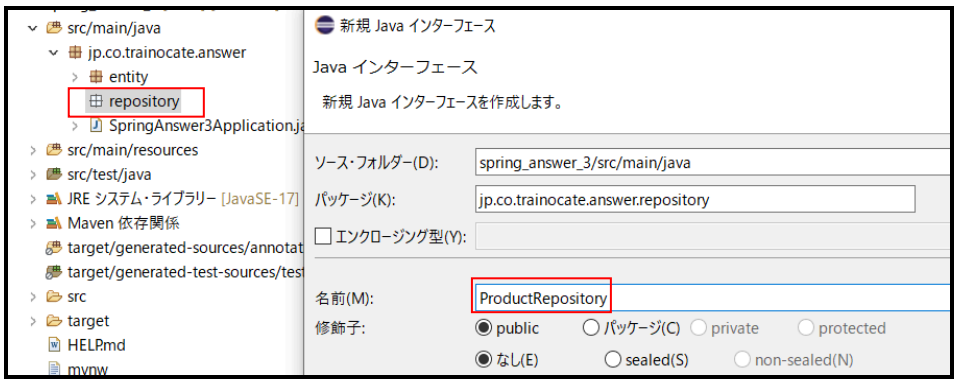

### リポジトリ（Repository）とは？

リポジトリは Spring Data JPA において、データベースとのやり取りを抽象化することができるインターフェースです。簡単に言うと、**データベースを操作できるリモコンのようなもの**です。

エンティティとリポジトリを組み合わせることで、**複雑なSQL クエリを書くことなく、Java を通じてデータベース操作を行うことができます。**リポジトリインターフェースを定義するだけで、Spring Data JPA は実行時にその実装を裏で自動的に生成し、開発者がデータベース操作に集中できるようにします。まるで、**リモコンに最初からついているボタンを押せば、テレビを簡単に操作できるのと似ています。**この節では、リポジトリの作成と基本的な使用方法について詳しく説明していきます。

## 【作業手順④】リポジトリの作成

エンティティと同様に、別途パッケージを作成します。クラスではなく、インターフェースですので、インターフェースとして作成します。名前は通常、エンティティ名の後ろに、[Repository]を結合することが多いです。




**…/repository/ProductRepository.java**

```
package jp.co.trainocate.answer.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import jp.co.trainocate.answer.entity.Product;

public interface ProductRepository extends JpaRepository<Product,
Integer>{ }
```

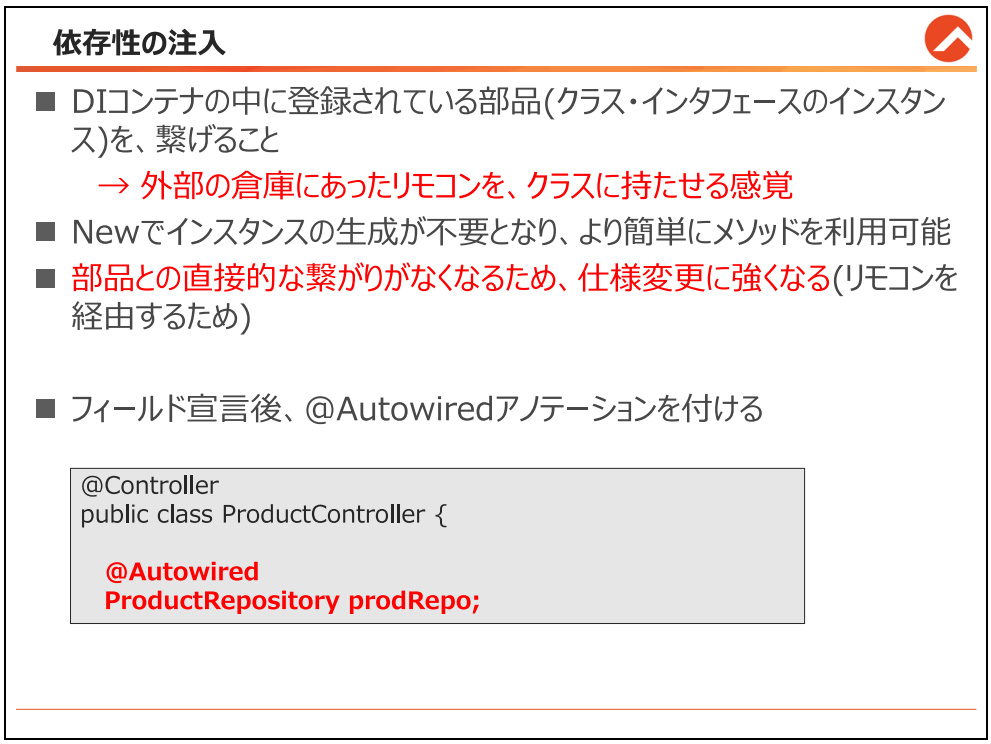
リポジトリを定義する際には、**JpaRepository という既存のインターフェースをextends で拡張**(継承)させます。

JpaRepository には、データベースのデータを検索したり保存したりするためのメソッドが最初から用意されています。それを**継承するだけで、開発者はSQL を記述しなくても、便利にデータベースの操作ができるようになります。**これはまるで、「リモコンによく使う電源On/Off やチャンネル・音量調整ボタンが最初から付いているため、テレビの中の複雑な処理を知らなくても簡単に操作できる」のと同じイメージです。

extends JpaRepository の後ろには、ジェネリクス(< >)を記載します。<Product, Integer>のように、**第1 引数にはエンティティ名を、第2 引数にはそのエンティティの@Id アノテーションを付けた主キーの型名**を記載します。

今までの内容をまとめると、Product というエンティティクラスがデータベースのテーブルを表し、そのクラスに対応するリポジトリを作ることで、テーブルのデータを簡単に追加、取得、更新、削除できるようになるわけです。 作成したリポジトリを実際の処理で使うためには、「依存性の注入」が必要になります。

## 3.3.1. 依存性の注入




### 依存性の注入とは

コントローラーからリポジトリを使用してDB 操作を行うためには、リポジトリを使えるようにする**「依存性の注入」**という準備が必要です。依存性の注入とは、**Spring のDI コンテナ（待機室）の中で待機しているクラスやインターフェースのインスタンスを、使いたいクラスに繋げること**です。

通常、Java プログラミングでA クラスのインスタンスをB クラスで使うためには、B クラス内で「A a = new A();」というオブジェクト生成の式を書く必要があります。しかし、**依存性の注入を行うことで、このNew 生成式を書かなくてもインスタンスを繋げて利用できる**ようになります。

第1 章で、Spring Boot のDI コンテナの中には「Bean（部品）」が自動で登録されていると説明しました。リポジトリのインスタンスもBean として自動登録されるため、必要な時にいつでもクラスに繋げることができます。

メリットはそれだけではなく、クラスのインスタンスを直接呼ばず、外部からのリモコンを使うだけなので、直接的な繋がりがなくなり、**部品の仕様変更にも強く柔軟なプログラムを実現できるようになります。**

では、具体的な注入方法を説明します。

```
@Autowired
ProductRepository prodRepo;
```

Spring では、いくつかの依存性の注入方法があります。一番簡単な方法は、上記のように、フィールドへ直接注入する方法があります。

インターフェースのフィールドを記載し、その上に[@Autowired]アノテーションを付けるだけで依存性の注入が完了し、そのインターフェースに紐づいているメソッドが使えるようになります。イメージとしては、**クラスに「ProductRepository 型」の「prodRepo」という名前のリモコンを持たせてあげた状態**です。

**【参考】その他の依存性の注入方法**

フィールドに@Autowired をつける方法は、とても便利ですが、単体テストの際に不便だというデメリットがあります。

ですので、下記の方法で書くこともあります。

**依存性の注入方法②**

```
public class ProductController {
    private final ProductRepository prodRepo;

    @Autowired
    public ProductController(ProductRepository prodRepo) {
        this.prodRepo = prodRepo;
    }
```

このように、フィールドを定数に変えた後に、コンストラクタを作成し、コンストラクタに@Autowired アノテーションを付ける依存性の注入の書き方もあります。こちらは単体テスト時も大丈夫ですので、よく使われます。

しかし、フィールドに指定する方法より、可読性が悪くなるため、下記のように、**lombok ライブラリを使って、簡略化させることも可能**です。

**依存性の注入方法③**

```
import lombok.RequiredArgsConstructor;

@Controller
@RequiredArgsConstructor
public class ProductController {
    private final ProductRepository prodRepo;
```

@RequiredArgsConstructor アノテーションを、クラス名に付けるだけで**依存性の注入のためのコンストラクタのコードを省略できます。**

## 3.3.2. コントローラーの基本URL 設定

今までは、メソッド単位でマッピングする URL を指定していましたが、通常、Spring プロジェクトには、複数の異なる機能を持つ、コントローラーが存在します。

例えば、**商品に関する機能を集めたコントローラーと、ユーザに関連する機能を持つコントローラーは、URL も区別しておかないと、URL の管理が難しくなります。**

そこで、コントローラーに基本 URL を指定しておくことで、URL の管理を便利にすることができます。

### ProductController の作成

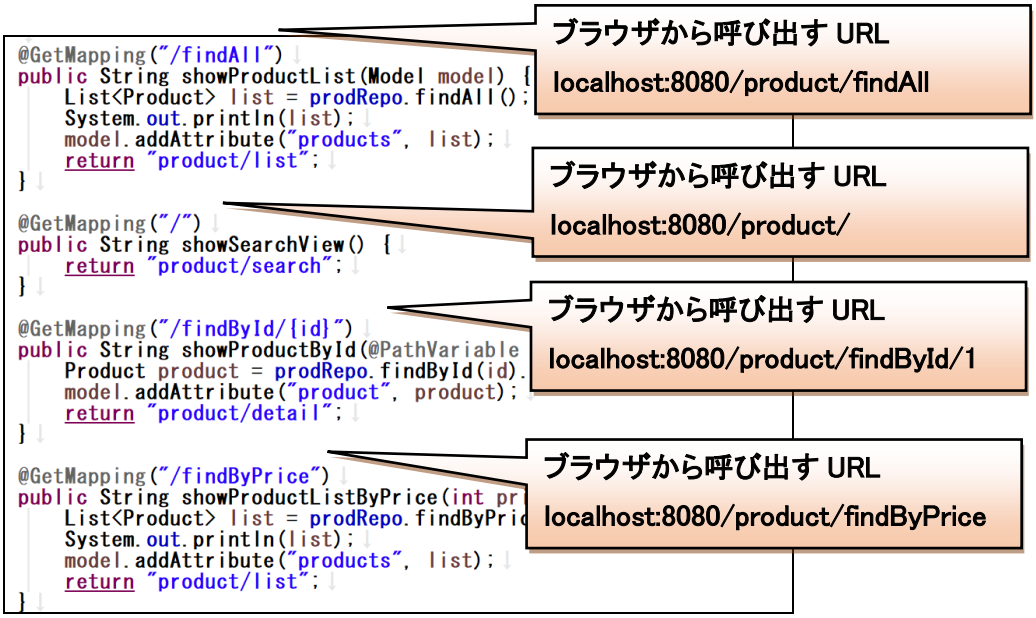
まずは、controller パッケージに[ProductController]クラスを作成します。

**…/controller/ProductController.java**

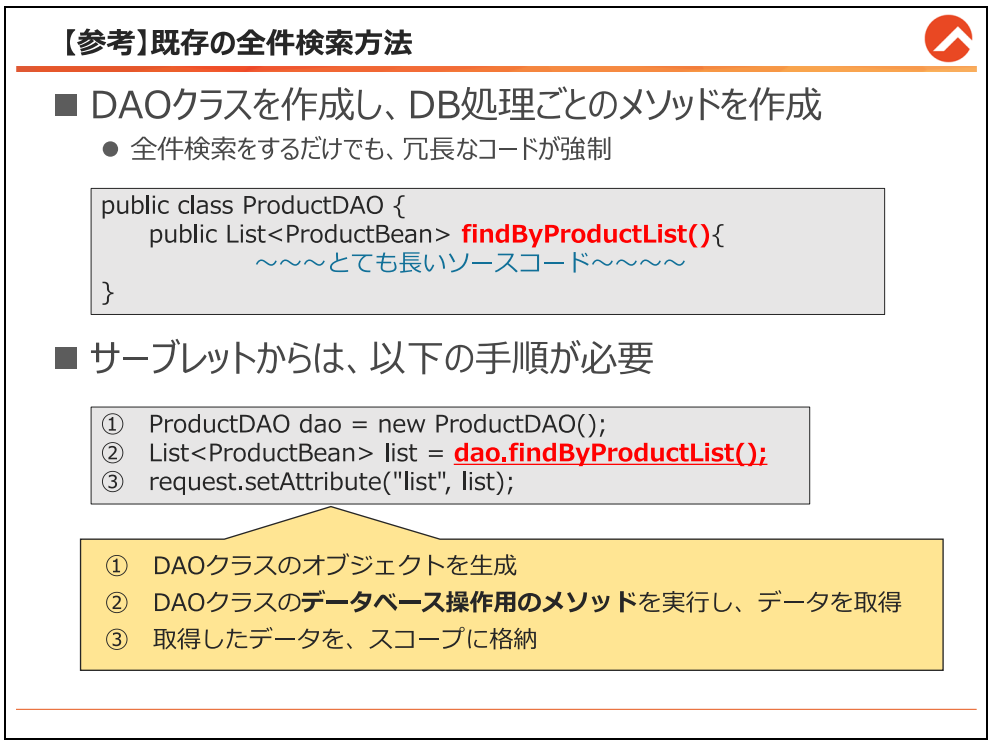
```
@Controller
@RequestMapping("/product")
public class ProductController {
```

class ProductController の上に、@RequestMapping(“URL”)というコードが書かれています。

これは、ProductController クラス全体に基本URL「/product」を設定しているコードです。RequestMapping でURL を指定すると、この**コントローラー内のすべてのメソッドは、基本URL に基づいてアクセスされます**。たとえば、ProductController クラスに作成されるメソッドのURL は、以下のように呼び出されることになります。



## 【 参考】既存の全件検索方法




データベースのアクセス情報を記載し、エンティティとリポジトリの定義まで終わったら、いよいよデータベース操作を開始できます。

では、本格的な説明の前に、まずは既存の Java データベースプログラミングとサーブレットでの書き方を確認しておきましょう。

### 【参考】サーブレットでデータベースの操作(全件検索)

サーブレットで、データベースから検索をするためには、別途DAO クラスを作成し、その中にデータベース操作をするためのメソッドを作成する必要がありました。

```
public class ProductDAO {

        public List<ProductBean> findByProductList(){
                String jdbcUrl = "jdbc:mysql://localhost/spring_
sample?"
                        + "serverTimezone=Asia/Tokyo";
                String userId = "spring_sample_user";
                String password = "trainocate";
```

```
        PreparedStatement pst = null;
        ResultSet rs = null;
        ArrayList list = new ArrayList();

        try (Connection con =
        DriverManager.getConnection(jdbcUrl, userId, password)) {
                String sql = "SELECT * FROM product";
                pst = con.prepareStatement(sql);
                rs = pst.executeQuery();
                while(rs.next()) {
                        ProductBean product = new ProductBean();
                        product.setId(rs.getInt("id"));
                        product.setName(rs.getString("name"));
                        product.setPrice(rs.getInt("price"));
                        list.add(product);
                }
        } catch (SQLException e) {
                e.printStackTrace();
        }
        return list;
}
```

一番簡単なSELECT 文なのに、これほどのソースコードが必要でした。
次は、サーブレット側のソースコードです。

```
@WebServlet("/Sample5")
public class Sample5 extends HttpServlet {
        protected void doGet(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

             //Product のDB 操作用クラスのProductDAO のオブジェクト生
成
                ProductDAO dao = new ProductDAO();

                //複数行を取得するので、リスト型でデータを取得
                List<ProductBean> list = dao.findByProductList();
```

```
        //取得したデータをJSP に送るため、リクエストスコープに格納
          request.setAttribute("list", list);

          //list.jsp へ遷移
          RequestDispatcher dispatcher =
              request.getRequestDispatcher("/index.jsp");
          dispatcher.forward(request, response);
}
```

サーブレットでは、以下の手順が必須でした。

① DAO クラスのオブジェクトを生成する。

② DAO クラスに定義されている**データベース操作用のメソッド**を実行し、データを取得する

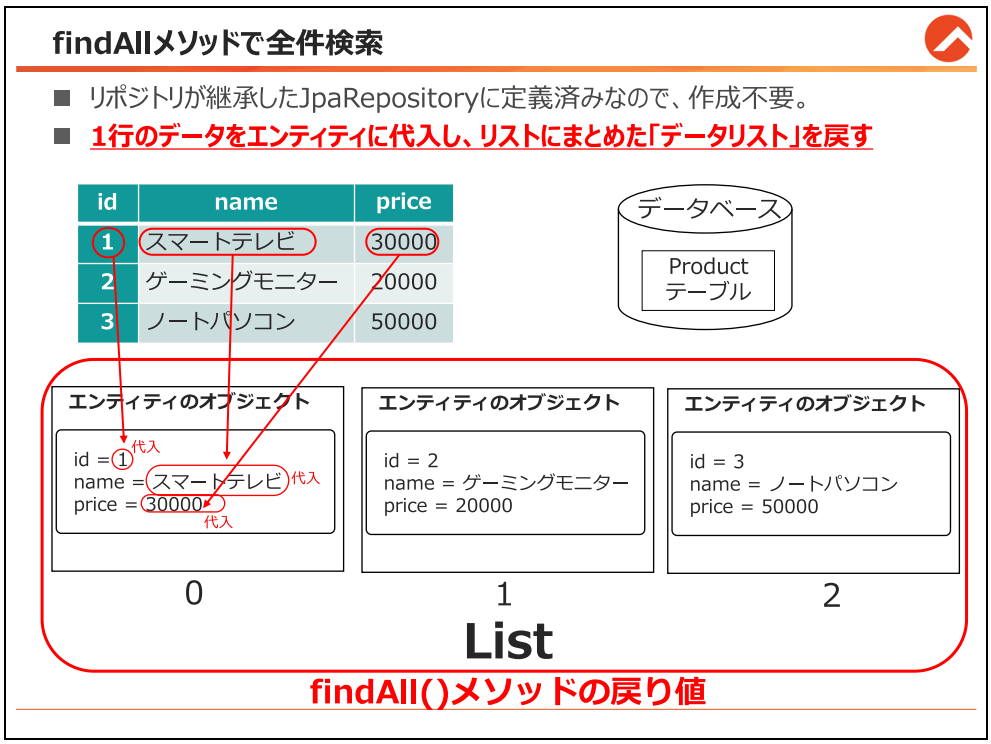
③ 取得したデータを、スコープに格納

④ JSP へ遷移

サーブレットの処理コード自体は、多くはないのですが、全件検索をするための、DAO クラスのメソッドのソースコードがかなりの量ですので、時間がかかっていました。

では、Spring Data JPA を使うと、どれほど簡単になるのか、今からご説明いたします。

## 3.4. 全件検索




**【作業手順①】findAll メソッドで全件データ取得**

**…/controller/ProductController.java**

```
 1:
 2:@Controller
 3:@RequestMapping("/product")
 4:public class ProductController {
 5:     @Autowired
 6: private ProductRepository prodRepo;
 7:
 8: @GetMapping("/findAll")
 9: public String showProductList(Model model) {
10: List<Product> list = prodRepo.findAll();
11: System.out.println(list);
12: model.addAttribute("products", list);
13: return "product/list";
14: }
15:}
```

3.3.2 で作成したProductController に、全件データの検索処理用のメソッドとして、[showProductList]メソッドを作成します。

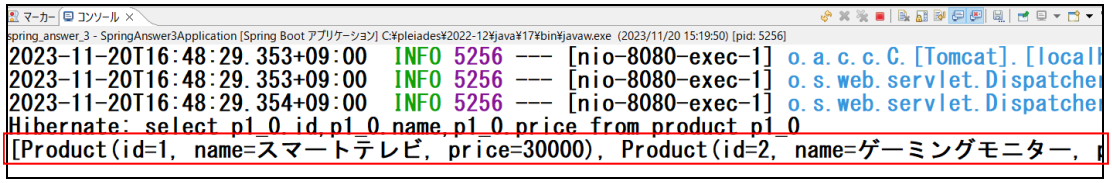
**5-6 行目**

```
5:    @Autowired
6: private ProductRepository prodRepo;
```

全件検索をするためには、ProductRepository から利用できる「findAll」メソッドを呼び出さないといけません。**ProductRepository のメソッドを簡単に利用できるように、ProductRepository のフィールドを宣言して、@Autowired で依存性の注入を**しておきましょう。この2 行のコードを書いておけば、[prodRepo]を使って、findAll メソッドを呼び出せます。

**10 行目 List<Product> list = prodRepo.findAll();**

findAll メソッドを実行すると、Product テーブル内の全データを、List<Product>の形で取得できます。**List のImport 先は「java.util」ですので、ご注意ください。**



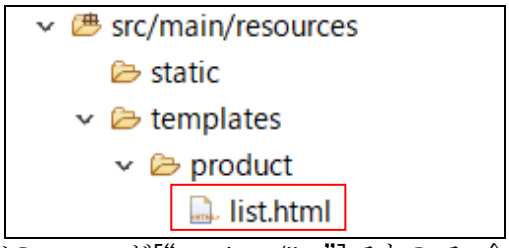

SELECT のSQL 文は、どこにも書いていませんが、データベースのproduct テーブルから商品データをしっかり取得しているのが確認できます。これがSpring Data JPA の凄いところです。エンティティにテーブルを忠実に再現した結果、**findAll というメソッドを実行するだけで、自動でデータの格納まで完了したリストを取得できます。**

**12-13 行目**

```
12: model.addAttribute("products", list);
13: return "product/list";
```

最後は、データの塊である、list をView へ出力させるために、model に格納します。このリストは、product が複数入っているので、キーは[products]にしました。遷移先は、product フォルダの、list.html を指しています。

## 【作業手順②】Thymeleaf でリストの出力




showProductList メソッドの return が[“product/list”]ですので、合わせてフォルダとファイルを作成します。

**…/resources/templates/product/list.html**

```
1:<h1>商品一覧</h1>
2: <table>
                                                    Spring Controller で
3: <tr>
                                                     model に格納した
4: <th>商品番号</th>
                                                      キー名を右に。
5: <th>商品名</th>
                                                    左は任意の変数名
6: <th>価格</th>
7: </tr>
8: <tr th:each="product: ${products}">
9: <td th:text="${product.id}"></td>
10: <td th:text="${product.name}"></td>
11: <td th:text="${product.price}"></td>
12: </tr>
13: </table>
```

**8 行目 <tr th:each="product: ${products}">**

th:each は、**Java の拡張for 文と似た使い方**をします。指定されたコレクション（ここでは products リスト）の各要素に対して、<tr> タグ（テーブルの行）を繰り返し生成します。

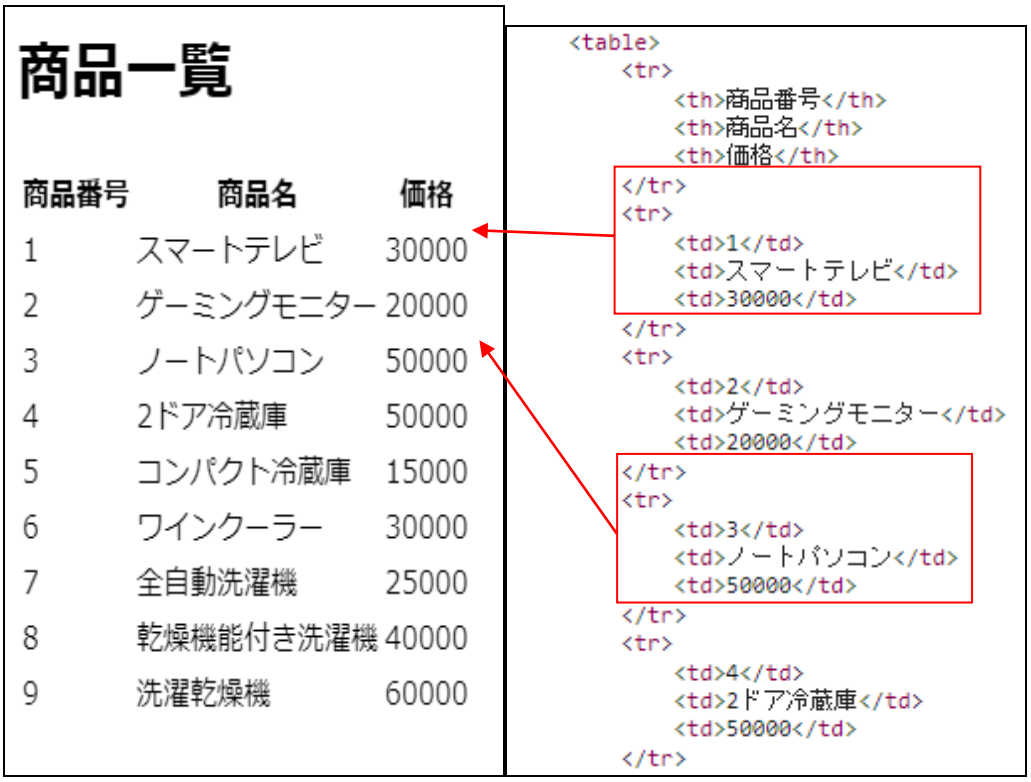
今回は、全部で9 個のデータがproducts リストの中に入っているので、products リストの要素が product 変数に9 回代入されます。**:を中心にして、右はController のmodel で格納したキー名、左はリストの中に格納されているものを意味する変数名を使います。**今回は、リストの中に、複数のproduct インスタンスが格納されているため、「product」としました。

**9-11 行目 <td th:text="${product.id}">、<td th:text="${product.name}">～～**

product 変数に1 行～9 行までのデータが順番に格納されていますので、「product.フィールド名」でデータを出力することができます。つまり、各product オブジェクトの id、name、price を<td>の中に9 回出力するソースコードが生成されます。

```
<tr th:each="product: ${products}">
      <td th:text="${product.id}"></td>
      <td th:text="${product.name}"></td>
      <td th:text="${product.price}"></td>
</tr>
```

<th:each>と<th:text>によって実際に生成されるHTML は、以下のようになります。




このテンプレートは、Spring Boot アプリケーションの ProductController のshowProductList メソッドによって返された products リストの内容を、ブラウザ上で見やすい表形式で表示するために使用されます。Thymeleaf のth:each を使用することで、リストの各要素に対してテーブルの行が自動的に生成され、リスト内のデータが簡単かつ効率的に表示されます。

### 動作確認

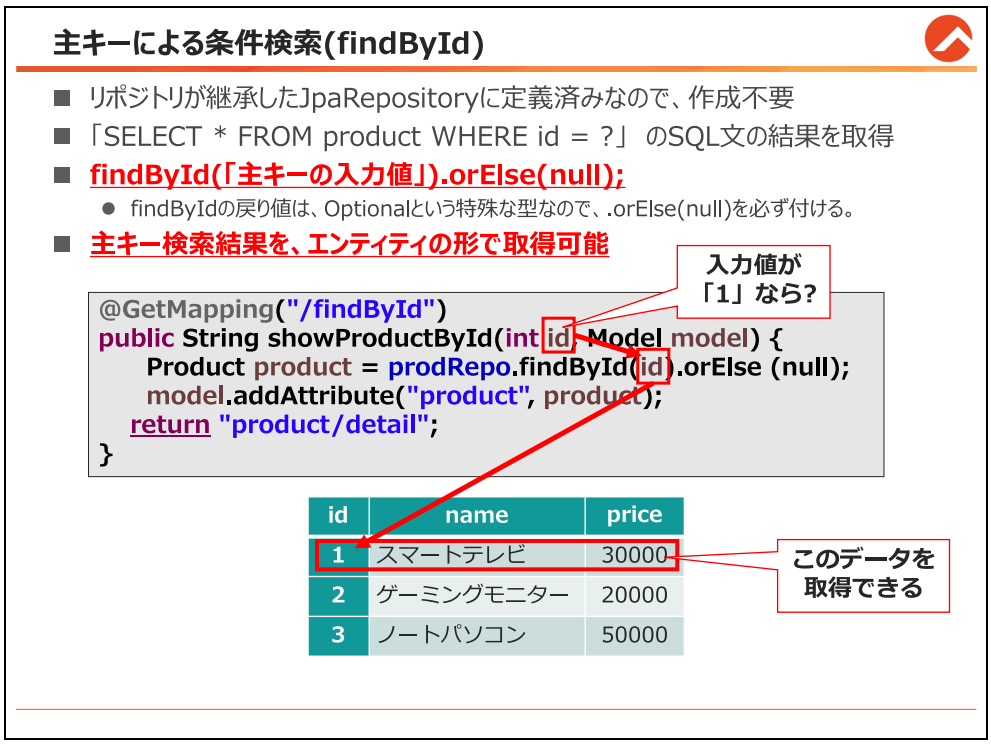
http://localhost:8080/product/findAll

Controller のURL マッピングが、[/product]メソッドのURL マッピングが[/findAll] になっているため、このようなURL を指定します。




th:each により、<tr>全体が繰り返しされるため、findAll メソッドで取得してView に送られてきたリストを1 行ずつ出力することができます。

## 3.5. 主キーによる条件検索




前の節では、findAll()メソッドを使って、全件検索ができるという内容をご説明しました。

この節では「SELECT * FROM WHERE id = ?」の SQL 文のような、主キーを使った条件検索の仕方について説明します。

## 検索用ページ作成

まずは、検索する ID を入力してもらうためのページを作成します。
そのためには、ページへ遷移させるためのメソッドを作成する手順が先です。




### 【作業手順①】遷移用のメソッド作成

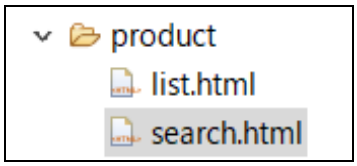
**…/controller/ProductController.java**

```
//showProductList メソッドの下に追記

@GetMapping("/")
        public String showSearchView() {
                return "product/search";
        }
```

/product を URL で入力すると、呼び出されるページ遷移用のメソッドを作成しましょう。
遷移先は、/product/search.html ですので、次は product フォルダ内に、search.html を作成します。

## 【作業手順②】検索入力View の作成




…/resources/templates/product/search.html

```
<!-- 省略 -->
<body>
        <h1>検索</h1>
        <h2>ID 検索</h2>
        <form th:action="@{/product/findById}">
                商品番号：<input type="text" name="id" />
                <input type="submit" value="検索">
        </form>
</body>
```

submit ボタンをクリックすると、[/product/findById]のメソッドを呼び出すように記載します。
送られる商品番号の name は、[id]です。

## 【作業手順③】主キー検索メソッドを作成

…/controller/ProductController.java

```
1://ProductController クラスのshowSearchView メソッドの下に追記
2:
3:@GetMapping("/findById")
4: public String showProductById(int id, Model model) {
5: Product product = prodRepo.findById(id).orElse(null);
6: System.out.println(product);
7: model.addAttribute("product", product);
8: return "product/detail";
9: }
```

**4 行目 showProductById(int id, Model model)**

View から送信される主キーのname 属性が「id」ですので、引数名を一致させることで、リクエストパラメータを取得します。

**5 行目 Product product = prodRepo.findById(id).orElse(null);**

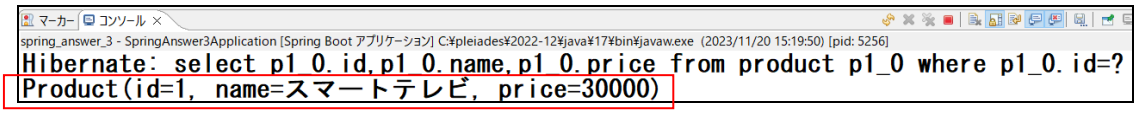
findById メソッドは、findAll と同じく、継承したJpaRepository の親に定義されているメソッドです。**引数には主キーを入れて、後ろに.orElse(null)を付けることで、主キー検索による、1 行の結果を取得**することができます。**.findById メソッドの戻り値は「Optional」という特殊な戻り値ですので、.orElse(null)をつけないとデータを取得できません。ご注意ください。**

**※.orElse(null)の代わりに、.get()** というメソッドもありますが、検索結果がない場合はnull ではなく、例外を出してアプリケーションを強制終了されてしまうので、お勧めしません。

findAll は、全件検索だったため、複数行が取得されることの前提でした。よって List<Product>型で受け取りました。**主キー検索は、必ず1 行のみ取得できるので、List での取得よりは、エンティティ単位での[Product product]で受け取るのが一般的です。**

**6 行目 System.out.println(product);**

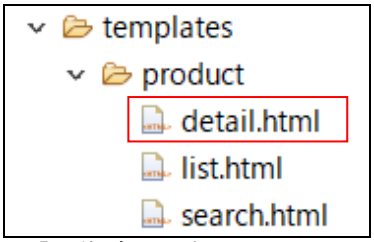
検索する商品番号に「1」を入れて実行させてみると、以下のように1 行のデータを取得できているのがわかります。




**7 行目 model.addAttribute("product", product);**

取得した1 行のデータは、Product エンティティの中に格納されているので、Viewに出力させるため、model に格納します。

**【作業手順④】商品詳細ページの作成**




product フォルダに、[detail.html]を作成します。

**…/resources/templates/product/detail.html**

```
<h1>商品詳細</h1>
        <table>
                <tr>
                        <th>商品番号</th>
                        <th>商品名</th>
                        <th>価格</th>
                </tr>
                <tr>
                        <td th:text="${product.id}"></td>
                        <td th:text="${product.name}"></td>
                        <td th:text="${product.price}"></td>
                </tr>
        </table>
```

showProductById メソッドのmodel に追加したキー名は、「product」ですので、th:text で各項目を表示させます。

**Model に格納されているのは、複数のデータではなく、商品1 個分のデータですので、th:eachを使う必要なく、そのままmodel のキー名で出力できます。**

**動作確認**

http://localhost:8080/product/ **←product の後ろに必ず/を付けてください**

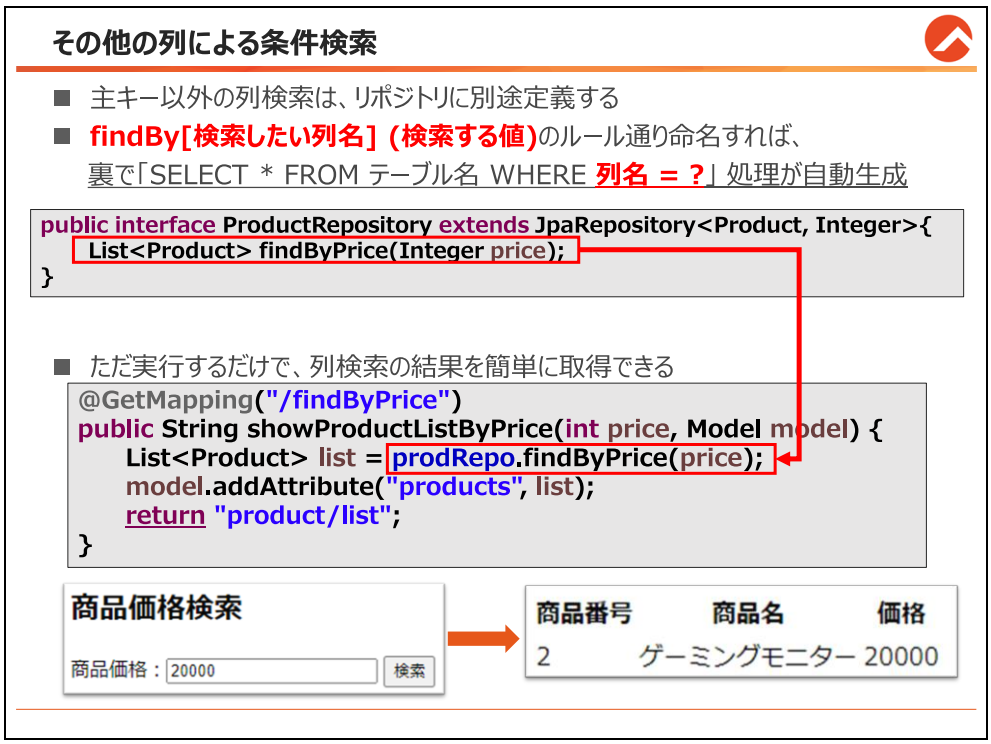



商品番号に、「1」と入力し、検索ボタンをクリックします。




商品詳細画面へ遷移し、商品番号1 番のデータが出力されたら、OK です。

## 3.5.1. 主キーではない列による条件検索



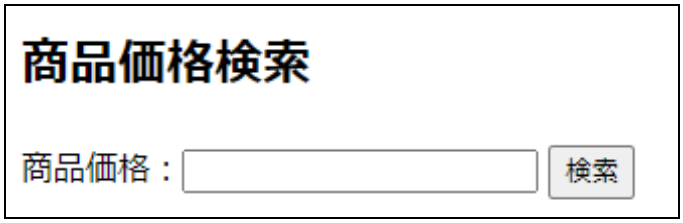

### 主キー以外の列による検索について

主キーは、汎用的に「id」という名称で使われますので「findById」というメソッド名が既に定義されていますが、主キー以外の普通の列による検索をする時を考えてみましょう。主キー以外の列名の検索は、列名を組み合わせたメソッドを開発者側から定義します。

Spring Data JPA を作った人は、利用者がどんな列名で検索をして来るかまでは予想できません。よって、**利用者自身がメソッド名に、それぞれの列名(正確には、エンティティ内のフィールド名)を付けることで、条件検索ができる**環境をリポジトリに用意しています。

この節では、**任意の列における検索メソッドであるfindBy〇ooo メソッドをリポジトリに追記**する方法についてご説明します。

**【作業手順①】価格による検索フォームの追加**




**…/resources/templates/product/search.html**

```
<!--ID 検索の下に追記　-->

<h2>商品価格検索</h2>
        <form th:action="@{/product/findByPrice}">
                商品価格：<input type="text" name="price" />
                <input type="submit" value="検索">
        </form>
```

前の節で作成した、search.html に、今度は商品価格検索フォームを追加しましょう。

リクエストパラメータは、price になり、submit ボタンをクリックすると、「/product/findByPrice」を URL マッピングしている Controller 側のメソッドが実行されます。

**【作業手順②】price 列で条件検索をするメソッドをリポジトリに追加**

**…/repository/ProductRepository.java**

```
//Product のimport に注意してください。
public interface ProductRepository extends JpaRepository<Product,
Integer>{
        List<Product> findByPrice(Integer price);
}
```

次は、リポジトリに抽象メソッドを書き込みます。

一見、インターフェースに抽象メソッドを記載しただけに見えますが、**リポジトリは、決まったルール通りのメソッド名を合わせるだけで、本当にその処理をするメソッドを裏で作ってくれます。**

例えば「**findBy[エンティティのフィールド名](入力値の変数)**」というルールに合わせれば、
SELECT * FROM product WHERE price = ○○;
という SQL 処理をするメソッドの処理が自動で完成されます。

product テーブルには**同じ価格を持つ商品が複数存在する可能性があるため、このメソッドの戻り値はリスト形式**になっています。もし、検索結果が複数行にならない条件検索をしたいなら、戻り値を「Product」のような、エンティティにすれば良いです。**出力の形を、戻り値を使って選べる**ということです。

findBy 列名 のメソッドを定義する際に注意しないといけないのは、**エンティティに定義されているフィール名と列名に1 文字でも誤字があったら、アプリケーションが起動しなくなります**ので、特にリポジトリを定義する時は、小文字・大文字や誤字に注意しましょう。

**例）findByprice / findbyPrice /FindByPrice / findByPlice→ NG**

**【作業手順③】showProductListByPrice メソッドを作成し、findByPrice のメソッドを実行**

**…/controller/ProductController.java**

```
1://showProductById メソッドの下に追記
2:
3:@GetMapping("/findByPrice")
4: public String showProductListByPrice(int price, Model model) {
5: List<Product> list = prodRepo.findByPrice(price);
6: System.out.println(list);
7: model.addAttribute("products", list);
8: return "product/list";
9: }
```

**5 行では、リクエストパラメータprice の値を取得し、prodRepo.findByPrice メソッドを実行しました。**

findAll メソッドを実行した時と、出力結果は同じくList になるメソッドですので、6 行以降は showProductList メソッドと全く同じです。

## 動作確認

http://localhost:8080/product/ ←product の後ろに必ず/を付けてください

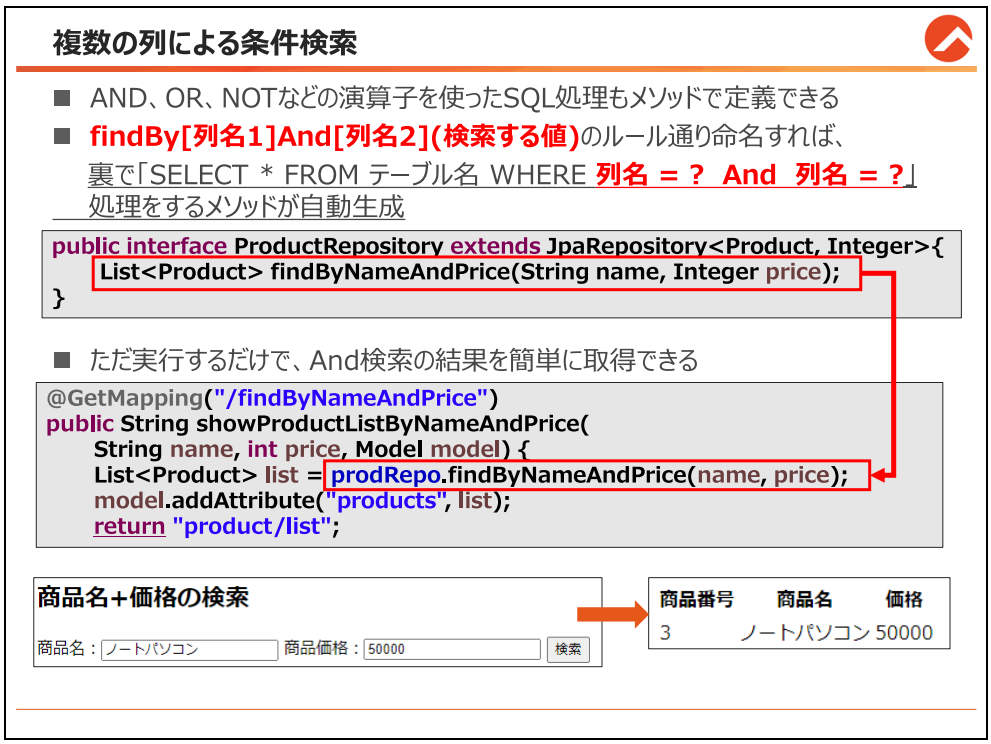



商品価格フォームに、[20000]と入力し、検索ボタンをクリックします。




商品一覧画面へ遷移し、価格が20000 のデータが出力されたら、OK です。

## 3.5.2. 複数の列による条件検索



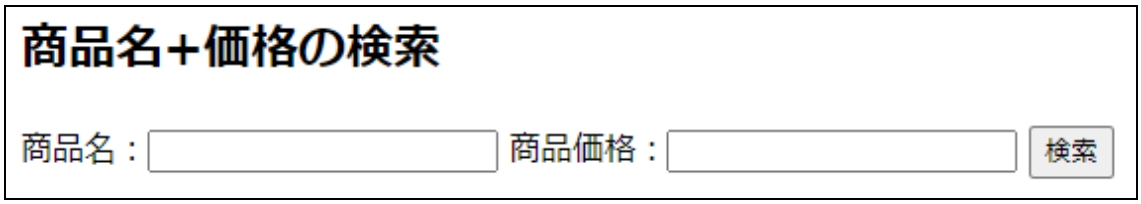

### 複数の列による検索について

Spring Data JPA では、「AND」や「OR」「NOT」などの演算子を使ったSELECT 文もメソッドとして定義できるようになっています。

では、「複数の列検索用のメソッド」を定義する方法についてご説明します。

findBy[エンティティのフィールド名]と同様に、リポジトリに追記しましょう。

### 【作業手順①】商品名と価格の検索フォームの追加




…/resources/templates/product/search.html

```
<!--商品価格検索の下に追記 -->

<h2>商品名+価格の検索</h2>
        <form th:action="@{/product/findByNameAndPrice}">
                商品名：<input type="text" name="name" />
                商品価格：<input type="text" name="price" />
                <input type="submit" value="検索">
        </form>
```

前の節で作成した、search.html に、今度は商品名と商品価格両方を使った検索フォームを追加しましょう。

リクエストパラメータは、name と price になり、submit ボタンをクリックすると、「/product/findByNameAndPrice」を URL マッピングしている Controller 側のメソッドが実行されます。

### 【作業手順②】name とprice 列で条件検索をするメソッドをリポジトリに追加

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product,
Integer>{
        List<Product> findByPrice(Integer price);
        List<Product> findByNameAndPrice(String name, Integer price);
}
```

次は、リポジトリに追加の抽象メソッドを書き込みます。

**今回のルールは、「findBy[エンティティのフィールド名 1]And[エンティティのフィールド名 2](入力値 1 の変数, 入力値 2 の変数)」です。**

SELECT * FROM product WHERE name = ○○ AND price = △△;

という SQL 処理をするメソッドが裏で完成されます。

product テーブルには同じ名前で同じ価格を持つ商品が複数存在する可能性があるため、このメソッドの戻り値はリスト形式になっています。もし、検索結果が複数行にならない条件検索をしたいなら、戻り値を「Product」のような、エンティティ型にすれば良いです。

**【作業手順③】showProductListByNameAndPrice メソッドを作成し、findByNameAndPrice メソッドを実行**

**…/controller/ProductController.java**

```
1://showProductListByPrice メソッドの下に追記
2:
3:@GetMapping("/findByNameAndPrice")
4: public String showProductListByNameAndPrice(String name, int
price,
                  Model model) {
5: List<Product> list = prodRepo.findByNameAndPrice(name, price);
6: System.out.println(list);
7: model.addAttribute("products", list);
8: return "product/list";
9: }
```

5 行では、リクエストパラメータname とprice の値を取得し、prodRepo. findByNameAndPrice メソッドを実行しました。

もし、入力値が更に増えるなら、Form クラスを使ってパラメータ取得しても良いでしょう。

6 行以降はshowProductList メソッドと全く同じです。

このようなAND 検索は、ログインID とパスワードのチェックの時、非常に使われる検索の仕方ですので、是非活用してください。

**AND 以外にも、OR やNOT も同じ方法でメソッドを定義すれば、その処理のメソッドが裏で自動生成されますので、とても便利です。**

参考として、以下のように複数の演算子を組み合わせるのもできます。

**例：findByNameOrPriceAndId**

### 動作確認

http://localhost:8080/product/ ←product の後ろに必ず/を付けてください




フォームに、[ノートパソコン]と、[50000]と入力し、検索ボタンをクリックします。




商品一覧画面へ遷移し、入力値が商品名と価格に一致しているデータが出力されたら、OKです。

## 3.5.3. その他の条件検索



その他の条件検索

| キーワード | 記述形式 | SQLでの機能 | メソッド定義の例 |
|---|---|---|---|
| Or | findBy[フィールド名1]Or[フィールド名2] | 論理演算子「OR」 | List<Product> findByIdOrPrice(int id, int price); |
| Like | findBy[フィールド名]Like　又は findBy[フィールド名]Containing | 述語「LIKE」 | List<Product> findByNameContaining (String name); |
| IsNull | findBy[フィールド名]IsNull | 演算子「IS NULL」 | List<Product> findByPriceIsNull(); |
| IsNotNull | findBy[フィールド名]IsNotNull | 演算子「IS NOT NULL」 | List<Product> findByPriceIsNotNull(); |
| OrderBy | findBy[検索対象のフィールド名]OrderBy[整列の基準となるフィールド名]ASC or DESC | ORDER BY 句 | List<Product> findByNameOrderByPriceDesc (String name); |
| GreaterThan | findBy[フィールド名]GreaterThan | 演算子「>」 | List<Product> findByPriceGreaterThan(int price); |
| LessThan | findBy[フィールド名]LessThan | 演算子「<」 | List<Product> findByPriceLessThan(int price); |
| Between | findBy[フィールド名]Between | 演算子「BETWEEN」 | List<Product> findByPriceBetween(int start, int end); |
| **Count** | CountBy[フィールド名] [条件] | 集合関数COUNT | int countByPriceLessThan(int price); |

Spring Data JPA で使用できるメソッドの命名規則を表形式でまとめると、以上のようになります。これらの規則を使うことで、Java のメソッド名を通じて SQL クエリに相当する機能を実現できます。

最後の行の Count については、findBy～ではない定義方法です。該当の条件での検索結果が何件なのか、整数で取得できます。

関数を使うなど、普通のSELECT文のとは異なる場合は、必ずしもfindByで始まるわけではなく、他にも色んな種類の定義方法がありますので、調べましょう。

※Like の場合、findBy[列名]Like と定義して呼び出しの際は、引数の前後に「%」のようなワイルドカードを付ける必要があります。

**例）List<Product> list = prodRepo.findByNameLike("%" + name + "%");**

%を前後に付けないと不具合が起きるため、Spring 公式ドキュメントでは、findBy[列名]Containing を推奨しています。

※全件検索を価格順(降順)で並び替えする時

**例）List<Product> list = prodRepo.findAllByOrderByPriceDesc();**

### その他のメソッドの例

```
public interface ProductRepository extends JpaRepository<Product,
Integer>{
        List<Product> findByPrice(Integer price);
        List<Product> findByNameAndPrice(String name, Integer price);

        // その他のメソッドの例

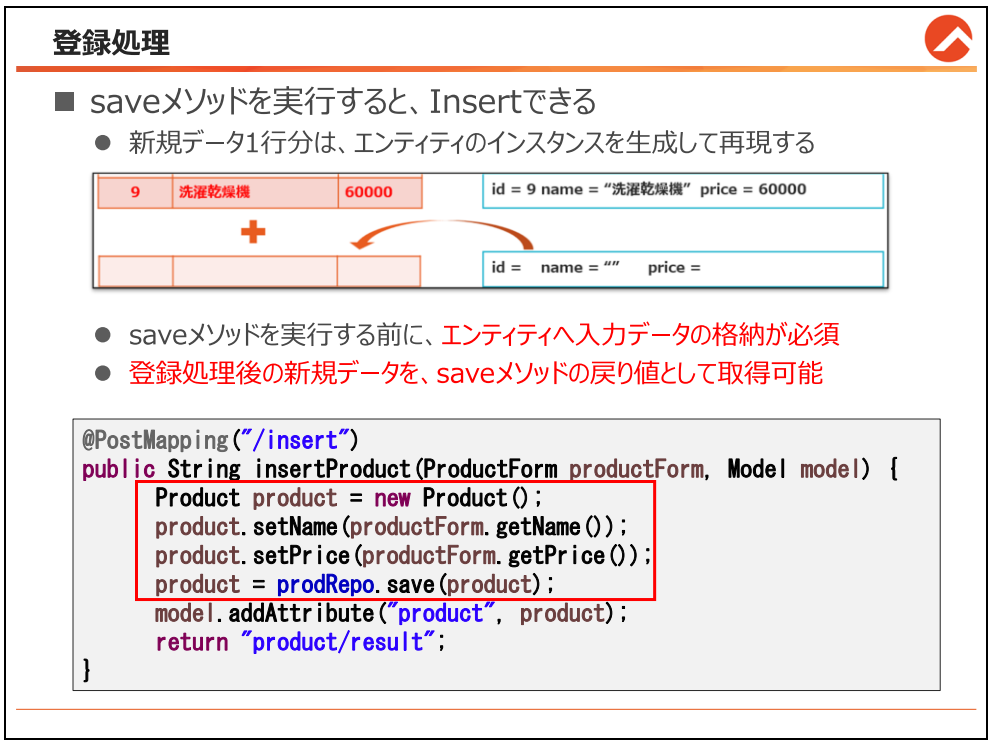
        // SQL: SELECT * FROM product WHERE id = ? OR price = ?;
        List<Product> findByIdOrPrice(int id, int price);
        // SQL: SELECT * FROM product WHERE name <> ?;
        List<Product> findByNameNot(String name);
        // SQL: SELECT * FROM product WHERE name LIKE ?;
        List<Product> findByNameLike(String name);
        // SQL: SELECT * FROM product WHERE price IS NULL;
        List<Product> findByPriceIsNull();
        // SQL: SELECT * FROM product WHERE price IS NOT NULL;
        List<Product> findByPriceIsNotNull();
        // SQL: SELECT * FROM product WHERE name = ? ORDER BY price
DESC;
        List<Product> findByNameOrderByPriceDesc(String name);
        // SQL: SELECT * FROM product WHERE price > ?;
        List<Product> findByPriceGreaterThan(int price);
        // SQL: SELECT * FROM product WHERE price < ?;
        List<Product> findByPriceLessThan(int price);
        // SQL: SELECT * FROM product WHERE price BETWEEN ? AND ?;
        List<Product> findByPriceBetween(int start, int end);
        // SQL: SELECT COUNT(*) FROM product WHERE price < ?;
        int countByPriceLessThan (int price);

}
```

どんな列を使い、どんな条件を使うかを考えて、自由自在の組み合わせで、異なる SQL 処理を作成できますので、単純なデータベースの操作に使う場合は、高い作業効率を期待できます。

しかし、結合やサブクエリなどを使う複雑なSQL 文については、JPQL やMyBatis のような別のライブラリを利用して、SQL 文を直接実行する方法を採用する場合もありますので、参考にしてください。

## 3.6. 登録処理




### 新規データ登録について

今までは、検索関連のデータベース操作を行う方法について学びました。次は、データを新しく登録する方法について学びます。

既存のデータベースプログラミングだと、登録も手間がかかるのですが、Form クラスと、Spring Data JPA の「save」メソッド使うことで、より簡単に登録作業ができます。

### データ入力フォーム作成

まずは、ユーザから登録データを入力してもらうためのView を作成するところからです。




### 【作業手順①】入力View 遷移用メソッドの定義

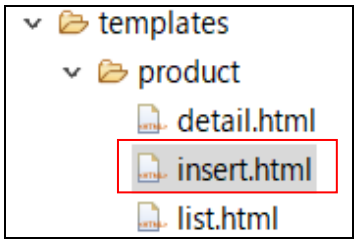
**…/controller/ProductController.java**

```
//findByNameAndPrice メソッドの下に追記

@GetMapping("/insert")
        public String showProductInsertView() {
                return "product/insert";
        }
```

まず、product フォルダのinsert.html へ遷移させるだけの、View 遷移用メソッドを作成します。product/insert のURL がリクエストされると、このメソッドにより、入力ページへ遷移するようにします。

## 【作業手順②】入力View 作成




**…/resources/templates/product/insert.html**

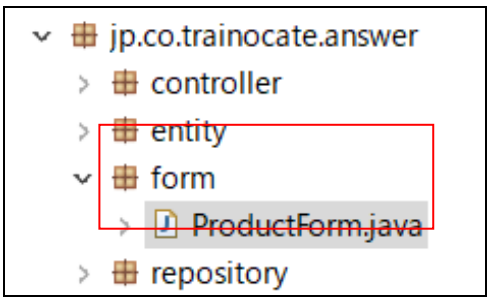
```
<!--<body>前は省略-->
<body>
        <h1>商品登録</h1>
        <form th:action = "@{/product/insert}" method = "POST">
        <p>商品番号：(自動割り当て)</p>
        <p>商品名　：<input type = "text" name = "name"></p>
        <p>商品価格：<input type = "text" name = "price"></p>
        <input type = "submit" value = "登録">
        </form>
</body>
```

th:action 属性値にご注目ください。Viewへ遷移する時も、「/product/insert」で、submit ボタンをクリックした時も、「/product/insert」にしていますが、POST 形式でリクエストを送っているため、別々のメソッドを呼び出すことができます。

商品登録(id)は、自動割り当てになっているため、入力しないようにします。

## 【作業手順③】ProductForm の定義

登録データは、通常複数のデータを扱うため、2 章でご紹介しましたForm クラスを利用すると便利です。現在、Product エンティティはあるものの、ProductForm が定義されていませんので、定義します。

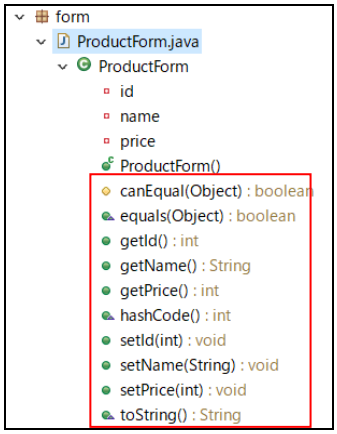



**…/form/ProductForm.java**

```
1:package jp.co.trainocate.answer.form;
2:
3:import lombok.Data;
4:
5:@Data
6:public class ProductForm {
7: private Integer id; //商品番号
8: private String name; //商品名
9: private Integer price; //商品価格
10:}
```

商品関連のデータを扱うために、[id]と、[name], [price]をフィールドとして持つForm クラスを作成しました。[id]は登録時には使われませんが、他の処理で使われるので、書くべきです。

5 行の@Data ですが、エンティティの定義の際にご説明しました通り、**@Data アノテーションをつけると、Getter&Setter メソッドは勿論、equals やtoString、hashCode メソッドが自動生成されますので、とても便利です。**



## 【作業手順④】登録処理メソッドの定義

…/controller/ProductController.java

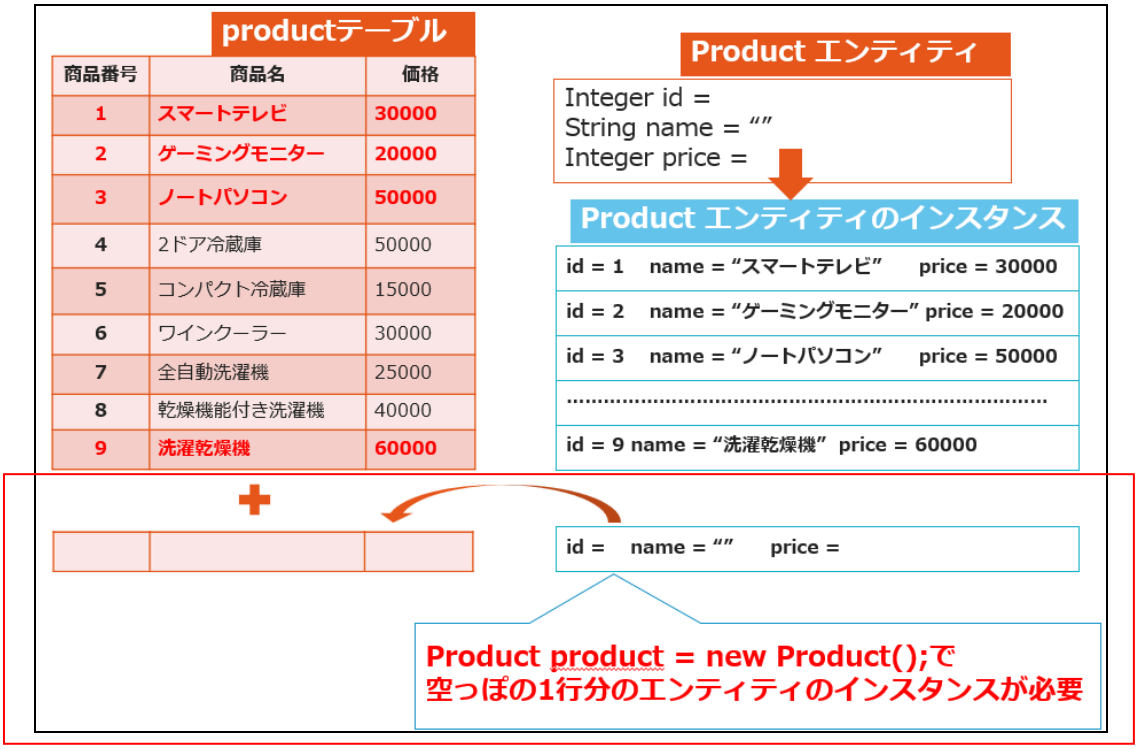
```
1://showProductInsertView()の下に追記
2:
3:@PostMapping("/insert")
4: public String insertProduct(ProductForm productForm, Model model)
{
5: Product product = new Product();
6: product.setName(productForm.getName());
7: product.setPrice(productForm.getPrice());
8: product = prodRepo.save(product);
9:              model.addAttribute("product", product);
10: return "/product/result";
11: }
```

**・4 行目: public String insertProduct(ProductForm productForm) {**

ユーザからの複数の入力値を受け取るために、productForm クラスをメソッドの引数に入れます。リクエストデータは、Form クラスのフィールドに自動でセットされます。

**・5 行目: Product product = new Product();**

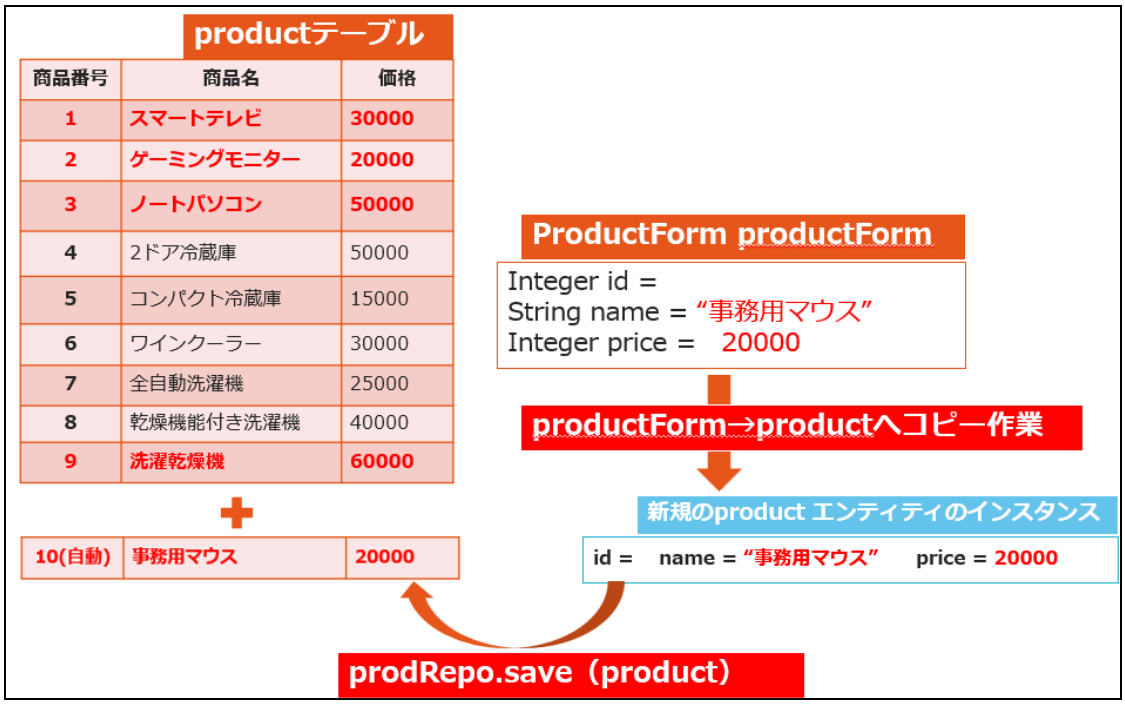
商品の登録をするためには、**予め商品1 個分をを表す、エンティティのインスタンスを生成する必要がありますので、new でProduct のオブジェクトを生成します。**



**・8 行目: product = prodRepo.save(product);**

先に説明します。Insert のSQL 文を実行させるためには、findAll と同じく、既にリポジトリに定義されている、[save]メソッドを実行すれば、登録処理が完了します。

**しかし、エンティティのインスタンスであるproduct は、5 行でオブジェクト生成はしたものの、データを入れないと、空っぽの状態のままになってしまいます。ユーザからのデータを受け取っているのは、product ではなく、4 行のProductForm の方ですので、データをコピーする必要があります。**




**・6～7 行目:**

**product.setName(productForm.getName());product.setPrice(productForm.getPrice());**

save メソッドの実行前に、Form インスタンス内のデータを、product へコピーさせます。

この作業により、空っぽだったproduct にも、ユーザからのデータが格納されました。

**・8-9 行目: product = prodRepo.save(product);model.addAttribute("product", product);**

**戻り値としては、登録完了した商品情報をエンティティのインスタンスとして取得**できます。ここには、自動割り当てされたid も含まれます。結果画面へ表示するため、model に格納します。

**・10 行目: return "/product/result";**本当に登録が完了したのか確認するために、商品データを出力する結果画面を表示させます。

…/resources/templates/product/result.html

```
1:<h1>商品・更新結果画面</h1>
 2: <table>
 3: <tr>
 4: <th>商品番号</th>
 5: <th>商品名</th>
 6: <th>価格</th>
 7: </tr>
 8: <tr>
 9: <td th:text="${product.id}"></td>
10: <td th:text="${product.name}"></td>
11: <td th:text="${product.price}"></td>
12: </tr>
13: </table>
14: <a th:href="@{/product/findAll}">商品一覧へ</a>
15:</body>
```

9 行目のproduct.id に注目してください。save メソッドの戻り値には、自動採番により割り当てされたID 番号も含まれているので、表示できます。

**動作確認**

http://localhost:8080/product/insert




任意の値を入力し、[登録]ボタンをクリックすると、登録後の全件検索結果が表示されます。




「商品一覧」をクリックすると、一覧画面に遷移します。




一覧でも新規データを確認できます。

### 【参考】ModelMapper で「Form→Entity のデータコピー」を簡単にする方法

今回の例では、入力値が2 つのみでしたので、コピー作業も2 行で終わりましたが、例えば、会員登録ページとかなら、10 個以上の値を扱います。その結果、10 行以上コードを書くことになり、冗長なコードになってしまいます。

その問題を解決するために、「ModelMapper」という外部ライブラリを利用すると便利です。

pom.xml

```
<dependency>
    <groupId>org.modelmapper</groupId>
    <artifactId>modelmapper</artifactId>
    <version>3.2.6</version>
</dependency>
```

まず、Mavenを使ってModelMapper 3.2.6を導入します。これはSpring Bootの管理対象外なので、バージョンを明記します。古い3.0.0ではなく、配布POMと同じ版を使ってください。

**…/controller/ProductController.java**

```
@PostMapping("/insert2")
public String insertProduct2(ProductForm productForm, Model model) {
    ModelMapper modelMapper = new ModelMapper();
    Product product = modelMapper.map(productForm, Product.class);
    product = prodRepo.save(product);
    model.addAttribute("product", product);
    return "product/result";
}
```

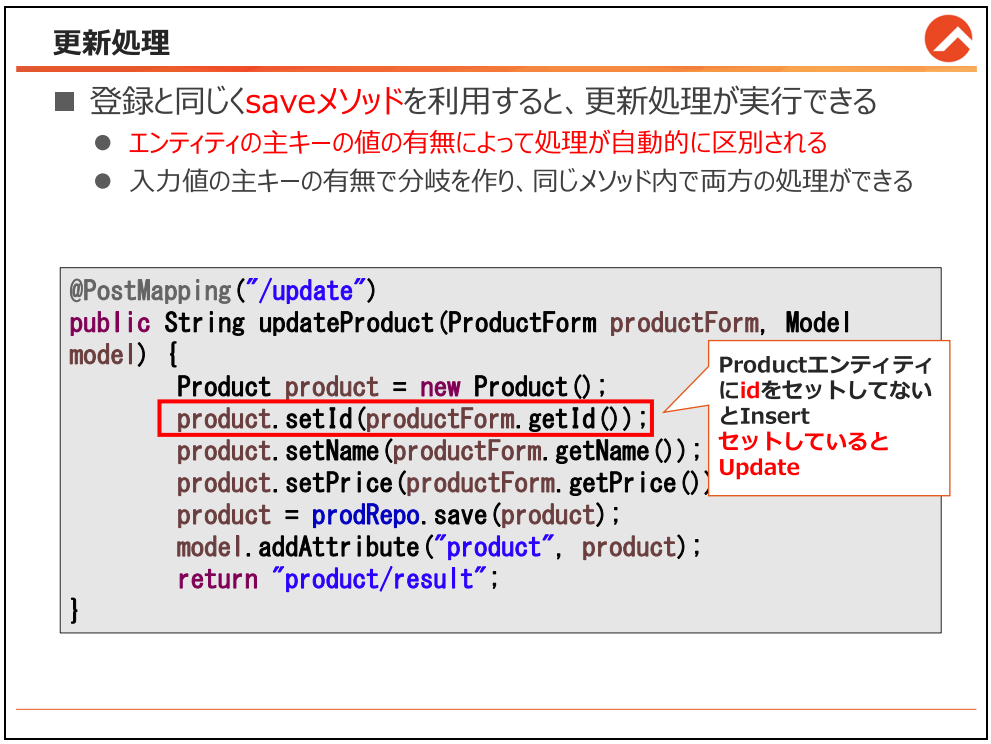


ここではメソッド内の new ModelMapper() でインスタンスを生成します。

modelMapper.map(Form名, Entity名.class) の戻り値をエンティティへ代入すると、同名フィールドの値をコピーできます。リクエストを受け取るForm引数と、変換に使うModelMapperは役割が異なります。

扱う入力値が多くなったら、是非こちらの方法を活用してみてください。

## 3.7. 更新処理




### save メソッドは更新処理も可能

登録と更新処理におけるSpring Data JPA の使い勝手の良さの一つは、両方の操作が同じ save メソッドを使っていることです。

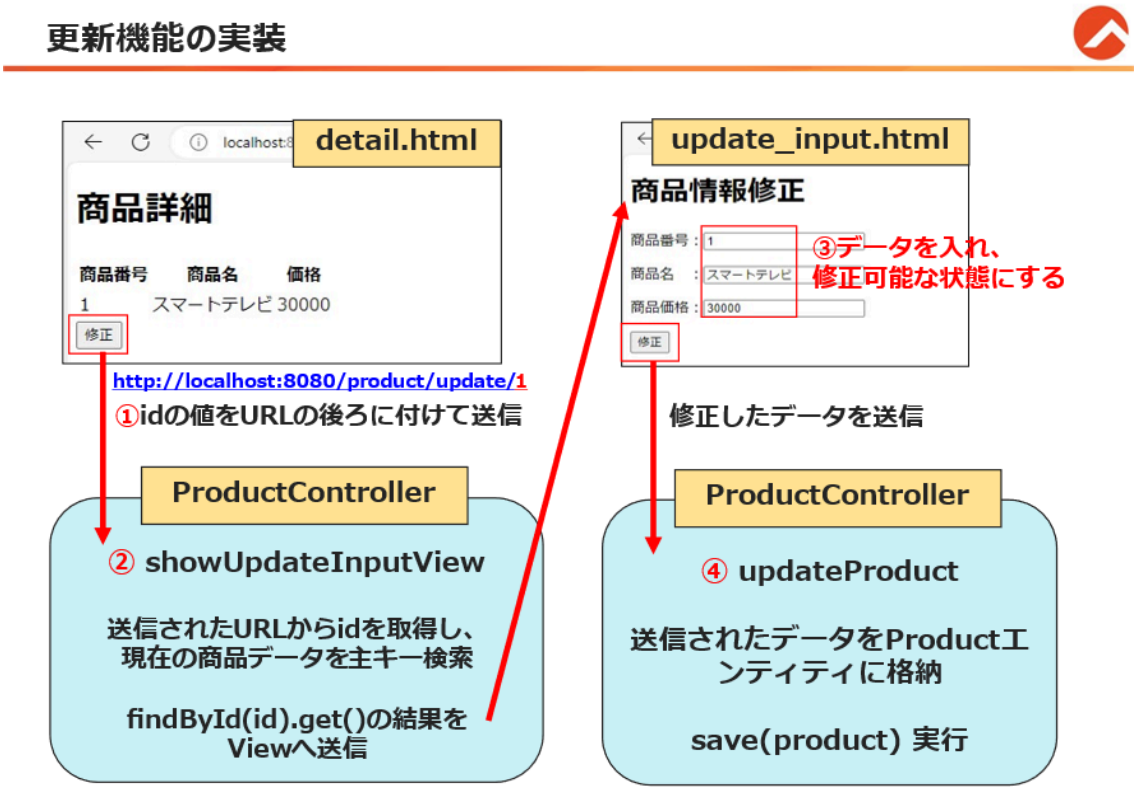
save メソッドを使用することで、新しいレコードの挿入(登録)だけでなく、既存レコードの変更(更新)も簡単に行えます。この柔軟性の秘密は、**エンティティの主キーの値の有無によって処理が自動的に区別される**点にあります。

**登録(INSERT)処理:** 新しいオブジェクト(エンティティ)を作成し、そのオブジェクトに主キーがまだ割り当てられていない場合(null またはデフォルト値)、save メソッドは新しいレコードをデータベースに挿入するSQL INSERT 文を実行します。

**更新(UPDATE)処理:** 主キーが割り当てられているエンティティをsave メソッドの引数に渡すと、Spring Data JPA はUPDATE 文を実行します。ここで重要なのは、主キーがすでに割り当てられているエンティティは、データベースにすでに存在しているとみなされるため、更新処理が行われます。

このアプローチは、コードの再利用性を高め、データベース処理をより直感的に行えるようにします。特に、**開発者が明示的にINSERT やUPDATE を意識する必要がなく、エンティティの状態に応じてSpring が適切な処理を自動的に行ってくれるため、開発効率が大幅に向上**します。エンティティのライフサイクル管理がこれほどシンプルになると、データモデルとビジネスロジックにより集中することができ、より洗練されたアプリケーションの開発が可能になります。

では、本格的な実装の前に、まずは流れを把握しましょう。




③の修正画面を開いた時、フォームには**現在の商品データが入力されている状態**です。それは、何もしなくても入力されているわけではなく、**事前に手順①と②を行った結果**です。

**更新処理は、他の処理とは違い、事前準備が必要だということを、是非理解した上で、進んでください。**

**【作業手順①】詳細画面に、更新ボタンを実装するように修正**

更新処理は、まず更新させたい対象を絞る必要があります。**対象を絞る時は、主キーのデータがあれば特定できますので、主キーのデータをもらう必要があります。**その方法の一つとして、本書では、検索から遷移できる「詳細画面」にボタンを配置させました。




**…/resources/templates/product/detail.html**

```
<!--</table>の下に追加 -->

<form th:action="@{/product/update/}+${product.id}">
        <input type = "submit" value = "修正">
</form>
```

URL にご注目ください。/product/update/ の後ろに、+ でproduct.id を付加しています。このように、Controller から取得した**データ（例えば、商品のID）をURL に組み込む**ことができます。これにより、特定の商品に関連したURL を動的に生成することが可能になります。

その結果、submit ボタンをクリックすると、以下のような **URL が動的に生成**されますので、どの商品を更新するかをController のメソッドへ明確に伝えることができます。

http://localhost:8080/product/update/1

では、Controller のメソッドで商品番号1 を取得する方法についてご説明します。

**【作業手順②】更新対象データの既存データ取得**

**…/controller/ProductController.java**

```
1://insertProduct メソッドの下に追加
2:
3:@GetMapping("update/{id}")
4:public String showUpdateInputView(@PathVariable int id, Model
model) {
5: Product product = prodRepo..findById(id).orElse(null);
6: model.addAttribute("product", product);
7: return "product/update_input";
8:}
```

**・3 行目: @GetMapping("update/{id}")**View から動的URL が送られる場合、同じく動的パラメータを指定する必要があります。書き方は、{ パラメータ名 } で記載します。

**・4 行目: public String showUpdateInputView(@PathVariable int id, Model model) {**

3 行で指定した動的URLの文字列と一致する引数名に**@PathVariable アノテーションをつけると、動的URL の値を取得**することができます。動的URL は、{id}なので、int id で合わせます。

5 行からは、更新対象の既存の値をfindById メソッドで取得+Model に格納し、update_input.html へ遷移させています。

**【作業手順③】修正フォーム（update_input.html）の作成**




showUpdateInputView メソッドのreturn により、既存のデータが表示されている画面へ遷移します。

**…/resources/templates/product/update_input.html**

```
<h1>商品情報修正</h1>
<form th:action = "@{/product/update}" method = "POST">
<p>商品番号：
<input type = "text" name = "id" th:value = "${product.id}"
readonly></input></p>
<p>商品名　：
<input type = "text" name = "name" th:value = "${product.name}"></p>
<p>商品価格：
<input type = "text" name = "price" th:value = "${product.price}"></
p>
<input type = "submit" value = "修正">
</form>
```

初めての構文として、「th:value」が登場しました。

**th:value は、Controller のmodel に格納されているデータをHTML 上で取得して、フォーム内に表示**するThymeleaf 独自の属性です。間違えて、ただの「value=“”」にすると、値を取得することができませんので、ご注意ください。

商品番号は主キーであり、変更してはいけませんので、readonly の属性を付与して、修正不可にしました。

「修正」のsubmit ボタンをクリックすると、フォーム内の入力データ値が全てリクエストパラメータとして/product/update のURL へ送信されます。

### 【作業手順④】修正処理メソッドの定義

**…/controller/ProductController.java**

```
1:@PostMapping("/update")
2:public String updateProduct(ProductForm productForm, Model model) {
3: Product product = new Product();
4: product.setId(productForm.getId());
5: product.setName(productForm.getName());
6: product.setPrice(productForm.getPrice());
7: product = prodRepo.save(product);
8:      model.addAttribute("product", product);
9: return "/product/result";
10:}
```

**・4 行目：product.setId(productForm.getId());**

登録の際は、主キーである「id」をProductエンティティ内にセットしていませんでしたが、更新処理の際は、必ず主キーのフィールドをセットさせます。

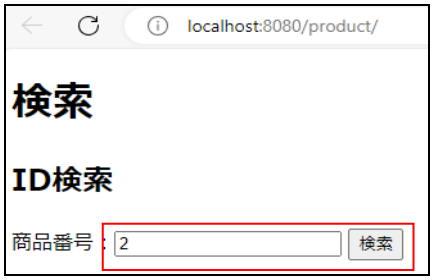
**・7 行目：product = prodRepo.save(product);**

save メソッドの引数に入れたエンティティのフィールドに、主キー(id)の値があったら、新しい値に上書きされる更新処理が実行されます。

その他のコードの解説は、登録と同じですので、割愛します。

## 動作確認

http://localhost:8080/product/ ←product の後ろに必ず/を付けてください




商品番号に、[2]を入力し、[検索]ボタンをクリックします。




商品番号2 番に対する商品データを確認し、[修正]ボタンをクリックします。




商品名と商品価格を任意の値で修正し、[修正]ボタンをクリックします。




商品一覧ページへ遷移し、2 番の商品が修正されていることを確認します。

### 【参考】登録と更新の処理を一つにする方法

**…/controller/ProductController.java**

```
1:@PostMapping("/insertOrUpdate")
2:public String insertOrUpdateProduct(ProductForm productForm) {
3: Product product = new Product();
4:     if (productForm.getId() != null) {
5:         // ID がある場合は、既存の商品として扱い、そのID を設定
6:         product.setId(productForm.getId());
7:     }
8:     product.setName(productForm.getName());
9:     product.setPrice(productForm.getPrice());
      // 新しい商品を追加または既存の商品更新
10:    product = prodRepo.save(product);
      // ～～割愛～～
```

上記のコードは、単純の分岐ですが、登録と更新を一つにまとめることができるので、開発効率を高めることができます。

## 3.8. 削除処理




削除については、更新と同じようなコードを使います。

### 【作業手順①】詳細画面に削除ボタンを追加




**…/resources/templates/product/detail.html**

```
<!-- detail.html に削除ボタンを追加 -->

<form th:action="@{/product/delete/}+${product.id}">
        <input type = "submit" value = "削除">
</form>
```

Update と同様に、th:action を活用して商品番号をURL に付けて送信します。

### 【作業手順②】削除処理のメソッドを定義

**…/controller/ProductController.java**

```
//省略

@GetMapping("/delete/{id}")
        public String deleteProduct(@PathVariable int id) {
                prodRepo.deleteById(id);
                return "redirect:/product/findAll";
        }
```

URL からid を取得し、どの商品を削除するかを決めます。
削除用のメソッドは、deleteById(id)を実行すれば、以下の削除SQL 文が実行されます。
「DELETE FROM product WHERE id = ?」
本当に削除が完了したのか確認するために、全件検索ページを呼び出したいところですが、全件検索は、html ファイルではなく、findAll メソッドを実行していました。

つまり、**returnでhtmlではなく、メソッドを呼び出すURLを指定する必要がある**ということです。
その時は、**return “redirect:/ [呼び出したいURL]”** の書き方にします。

**動作確認**

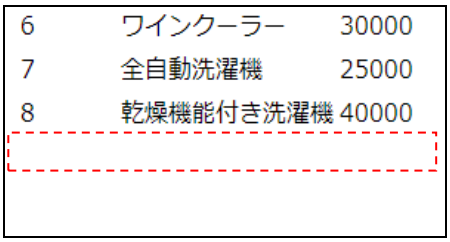
**http://localhost:8080/product/ ←product の後ろに必ず/を付けてください**




任意の商品番号を検索します。

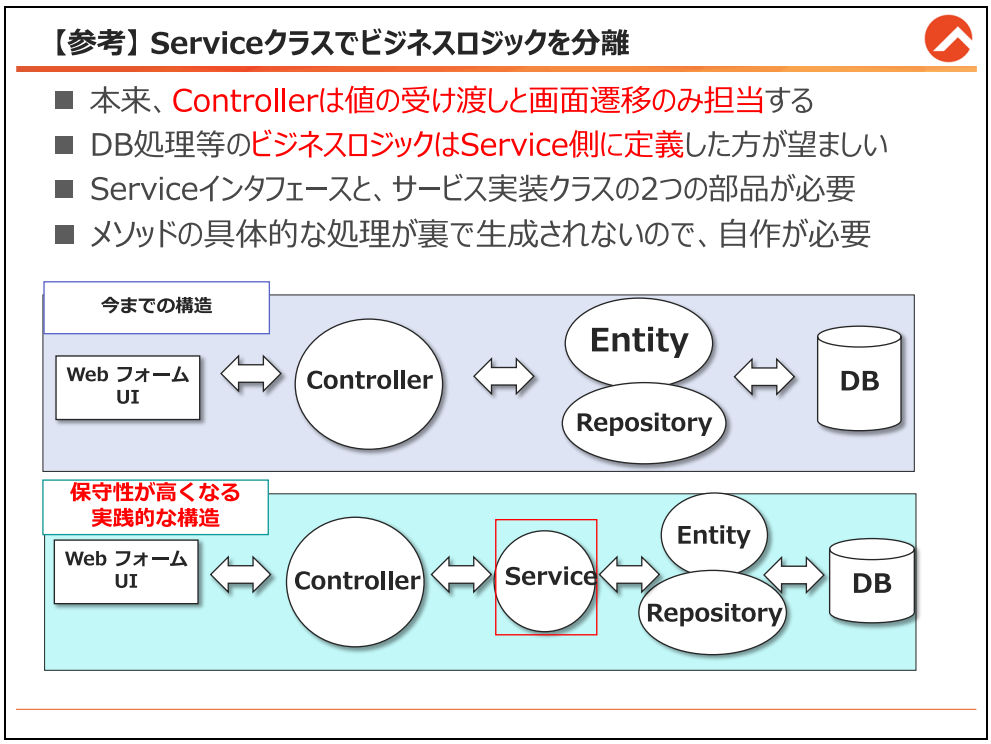



削除ボタンをクリックします。




商品が削除されたのを確認できればOK です。

## 3.9.【 参考】Service クラスでビジネスロジックを分離




これまでの例では、簡単に理解できるように、Controller クラスの中にデータベース処理などのビジネスロジックも書いていました。

しかし、本来Controller の役割は、データのやり取りと画面の移動にあります。より良い保守性を実現するために、実際の開発現場では**「Service インターフェース」と「Service 実装クラス」と呼ばれる専門の部品を2 つ作り**、ビジネスロジック（例えば、データベース操作）を集めます。

### Service クラスの目的と役割

コントローラーは主にHTTP リクエストの受け取りとレスポンスの送信、画面遷移の管理を行います。一方で、Service クラスはビジネスロジック（例えば、データの検証、計算、データベースの操作など）を担当します。Service クラスにビジネスロジックを集中させることで、コードの再利用性が高まり、変更や拡張が容易になります。また、コードが整理され、各クラスの役割が明確になるため、保守しやすくなります。

### サービスインターフェースとサービス実装クラスを分けるメリット

インターフェースと実装クラスの2 つに分けて作成する理由は、メリットがあるからです。まずは、以下の例えでイメージを持ってください。

**例え: テレビのリモコンと本体**

■ **サービスインターフェース: テレビの「リモコン」**

テレビの電源をつけたり、音量を調整したりする「入口」。

リモコンの操作方法は変わらない（電源ボタンを押せばテレビがつく）。

■ **サービス実装クラス: テレビの「本体」**

実際に映像や音を出す機械。

本体の仕組みは変わるかもしれない（電源の音が変わるとか）が、リモコンの使い方は変わらない。

**メリット①：柔軟性が高くなる**

■ **分けない場合:**実装が変わると、呼び出している全ての場所（Controller など）を修正しないといけない。

■ **分けた場合:**リモコン（インターフェース）を使って呼び出すため、実装（テレビ本体）が変わってもリモコンの使い方は変わらない。

**メリット②：実装を切り替えやすい**

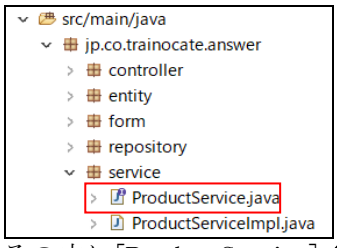
現場では、以下のような場面で実装を切り替えることがよくあります。

■ **データベース→外部API: データの取得元を変更したい。**

■ **動作確認用のダミー: テスト環境では、ダミーのデータを返すモッククラスを使いた**い。

このような場合、実装クラスを切り替えるだけで、テスト環境や新しい仕様に対応可能になりますので、インターフェースと実装クラスを分けるわけです。

**【作業手順①】サービスインターフェースの作成**




[service]パッケージを作成し、その中に[ProductService]**インターフェース**を作成します。

**…/service/ProductService.java**

```
package jp.co.trainocate.answer.service;

import jp.co.trainocate.answer.entity.Product;

public interface ProductService {
        public Product findProductById(int id);
}
```

学習のため、主キー検索の機能をする抽象メソッドのみ実装します。

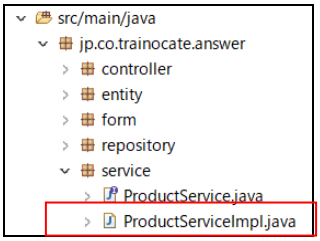
インターフェースは**リモコンだと思ってください。リモコンにはボタンが必要です。リモコンのボタンは抽象メソッドとして表現します。**

ですので、メソッドの書き方は、JPA のリポジトリに検索用のメソッドを定義していた時と似ています。この処理をするためには、どんな引数が必要そうか、どんな戻り値を戻せばいいのかを考えます。

リポジトリのような、メソッド名の規則は特にありません。どんなメソッドなのか意味が通じるメソッド名で大丈夫です。

**ただのボタンですので、具体的な処理は書かずに、引数と戻り値だけ決めた抽象メソッドとして作成**しておきます。

**【作業手順②】サービス実装クラスの作成**




サービスインターフェースで、リモコンのボタン(抽象メソッド)の定義が完了したら、**リモコンのボタンを押した時に実際に処理をする側の、「サービス実装クラス」をセットで作成**する必要があります。

JPA のリポジトリは、リモコンのボタンさえ作っておけば、実際の処理をするメソッドは裏で自動生成されましたが、**サービスの場合は、自作で抽象メソッド一つ一つの具体的な実装が必要**です。

サービス実装クラスは、サービスインターフェース名に+「Impl」を追加した名前に合わせるのが一般的です。

**…/service/ProductServiceImpl.java**

```
 1:import jp.co.trainocate.answer.entity.Product;
 2:import jp.co.trainocate.answer.repository.ProductRepository;
 3:import lombok.RequiredArgsConstructor;
 4:
 5:@Service
 6:@RequiredArgsConstructor
 7:public class ProductServiceImpl implements ProductService{
 8:
 9: //@Autowired は不要
10: private final ProductRepository prodRepo;
11:
12: @Override
13: public Product findProductById(int id) {
14: return prodRepo..findById(id).orElse(null);
15: }
16:}
```

**・5 行目：@Service**サービス実装クラスには、@Service アノテーションを付けます。忘れがちですので、注意してください。

**・6 行目：@RequiredArgsConstructor**

3.3.1「依存性の注入」の【依存性の注入方法③】でご紹介した書き方です。

9-10 行目のコードと連携します。単体テストのしやすさのために、サービス実装クラスではこの方法が推奨されます。

**・7 行目：implements ProductService**前の手順で作成したサービスインターフェースと繋げるために、implements します。

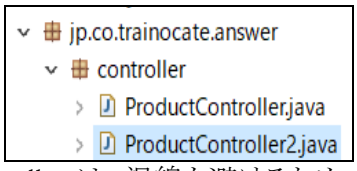
**・9 行-10 行目：private final ProductRepository prodRepo;**

DB 処理のためProductRepository を利用したいので、依存性の注入を行います。6 行目と合わせて記載することで、コンストラクタを自動生成できます。

**・13 行-14 行目：return prodRepo..findById(id).orElse(null);**

Controller に記載していたDB 処理部分だけ分離させ、DB から取得した商品データをそのままreturn しています。ProductService インターフェースで定義した抽象メソッドと同じ呼び方をするように、オーバーライドする必要があります。(リモコンのボタンを押したら、このメソッドの処理が行われるイメージ)

## 【作業手順③】Controller からService クラスを経由してDB 操作を行う




Service クラスを経由するController は、混線を避けるため、[ProductController2]で新規作成しました。

**…/controller/ProductController2.java**

```
 1:@Controller
 2:@RequestMapping("product2")
 3:
 4:public class ProductController2 {
 6:    @Autowired
 7: private ProductService prodService;
 8:
 9://～省略～
10:
11: @GetMapping("/findById")
12: public String showProductById(int id, Model model) {
13: Product product = prodService.findProductById(id);
14: model.addAttribute("product", product);
15: return "product/detail";
16:   }
```

**・2 行目: @RequestMapping("product2")**

Controller が変わったので、異なる基本URL を設定します。

**・7 行目 : private ProductService prodService;**

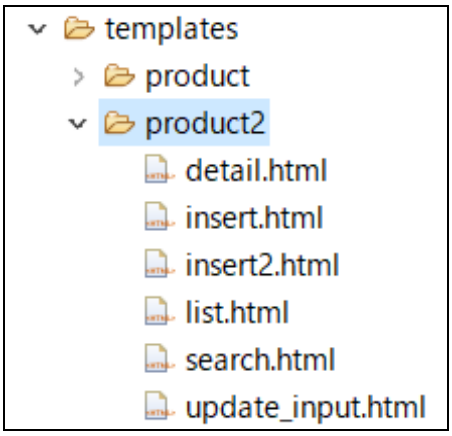
**DB処理は、ProductServiceの方で担当**するので、このクラスで利用できるように、依存性の注入を行います。既存のProductRepository の依存性の注入は不要になります。

**・13 行目 : Product product = prodService.findProductById(id);**

ここが一番大事です。今までは、Controller のメソッドの中で直接「prodRepo.メソッド」を実行する書き方でしたが、今回はServiceクラスに定義されているメソッドを実行させて、その戻り値を利用する書き方になりました。

「ProductService」インターフェース型のインスタンスからメソッドを呼び出していますが、「ProductServiceImpl」クラスに具体的な処理がオーバーライドされているので、実際は「ProductServiceImpl」のメソッドがポリモフィズムで実行されるようになります。

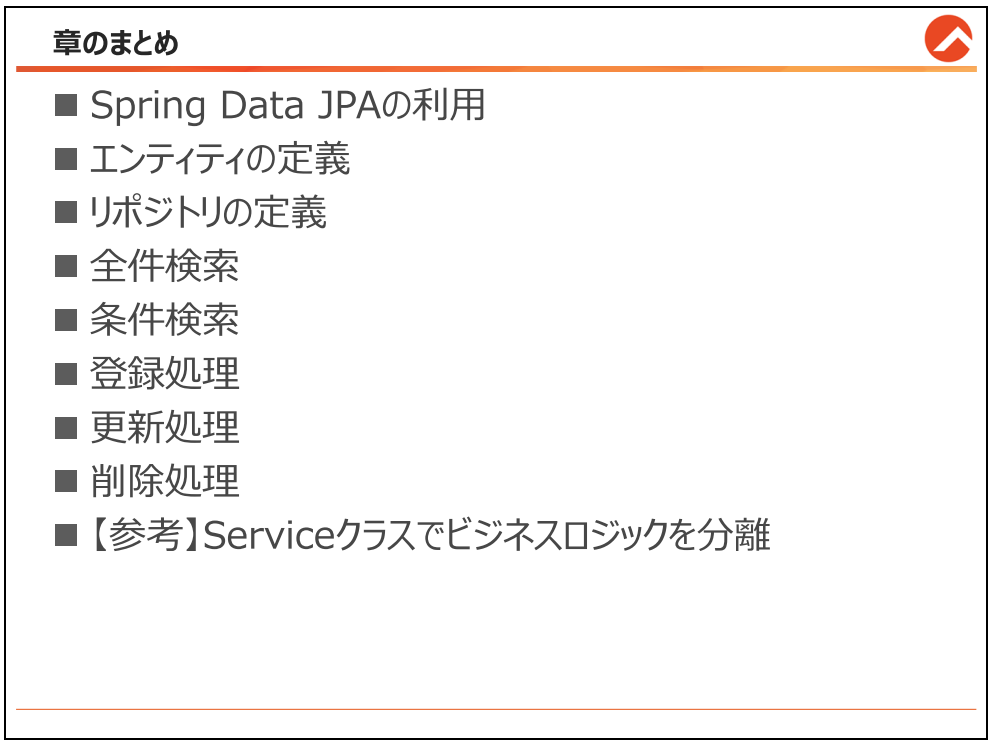
このような書き方をすることで、**Controller はあくまでデータの受け渡しの役割に集中させ、ビジネスロジック(DB 処理)はService クラスで行わせます。**




※基礎URL がproduct2 になるので、product2 フォルダに同じhtml を格納する必要があります。

※その他の例は、[ProductController2.java]と、[ProductService.java]、[ProductServiceImpl.java]を参照してください。

## 章のまとめ




この章では、Spring Data JPA を使った、データ操作方法についてご説明しました。